Lista de endpoints de la API Brújula Viajes

Ingeniería de Datos - Proyecto Brújula Viajes - Evidencias del backend

Antonio Gutierrez, David Rodriguez, David Pascagaza, Sara Torres

1. Introducción

Este documento describe los endpoints mínimos de la API REST del sistema Brújula Viajes, desarrollada con FastAPI, PyMongo y Pydantic sobre una base de datos MongoDB. Para cada endpoint se presentan el método HTTP, la ruta, las colecciones y reglas de la base de datos que intervienen, una descripción de su funcionamiento y la evidencia de la prueba. Las pruebas se ejecutaron en la documentación interactiva de la API (Swagger, ruta /docs) contra la base de datos cargada con los datos de prueba del proyecto; la evidencia 1 se ejecutó directamente en mongosh desde MongoDB Compass, como pide el enunciado.

La primera parte reúne las evidencias de las validaciones y reglas exigidas: el documento rechazado por el validador de esquema, la reserva rechazada por falta de cupo, la reserva cancelada que devuelve su inventario y los reportes B1 a B3. La segunda parte documenta la prueba de cada endpoint mínimo, agrupada por recurso.

2. Marco conceptual

2.1. Métodos HTTP

Método	**Uso en la API**
GET	Consultar información sin modificar datos.
POST	Crear un registro nuevo o ejecutar una acción (registrar un pago, cancelar una reserva).

Tabla 1: Métodos HTTP utilizados.

2.2. Códigos de respuesta

Código	Significado
200 OK	La consulta o la acción se realizó correctamente.
201 Created	El registro se creó correctamente.
404 Not Found	El registro solicitado no existe.
409 Conflict	Una regla de negocio rechazó la operación: no hay cupo suficiente, la reserva ya está cancelada o el pago supera el saldo.
422 Unprocessable Entity	Los datos enviados no cumplen las validaciones del DTO de Pydantic.

Tabla 2: Códigos de respuesta HTTP.

2.3. Flujo de una petición

Cuando un cliente (Swagger en /docs o el frontend en Streamlit) envía una petición, esta recorre las siguientes capas:

1. El router de FastAPI recibe la petición según su método y ruta.
2. El DTO de Pydantic (`XxxCreate`) valida los datos de entrada. Si no son válidos, se responde 422.
3. PyMongo traduce la operación a consultas y escrituras sobre las colecciones de MongoDB.
4. Las reglas de negocio se aplican en la base: escrituras condicionales atómicas (por ejemplo, descontar sillas solo si hay suficientes libres, con `$elemMatch` y `$inc`), índices únicos y validadores `$jsonSchema`. Si una regla se incumple, la operación se cancela y la API responde 409.
5. El DTO de respuesta (`XxxResponse`) da forma a los datos devueltos; el `_id` de MongoDB (ObjectId) sale como texto en el campo `id`.

Las reglas de negocio residen en la base de datos y no solo en Python: los validadores e índices únicos se cumplen incluso si los datos se modifican directamente desde mongosh o Compass, sin pasar por la API (evidencia 1).

3. Resumen de endpoints

Prueba	Método	Ruta	Descripción
backend-01	GET	`/clientes/`	Listar clientes
backend-02	POST	`/clientes/`	Registrar cliente
backend-03	GET	`/vuelos/`	Buscar vuelos
backend-04	POST	`/vuelos/`	Registrar vuelo
backend-05	GET	`/hoteles/`	Listar hoteles
backend-06	POST	`/hoteles/`	Registrar hotel
backend-07	GET	`/tours/`	Listar tours
backend-08	POST	`/tours/`	Registrar tour
backend-09	GET	`/paquetes/`	Listar paquetes vigentes
backend-10	POST	`/paquetes/`	Registrar paquete
backend-11	GET	`/reservas/`	Listar reservas
backend-12	POST	`/reservas/`	Crear reserva
backend-13	GET	`/reservas/{codigo}`	Obtener detalle de una reserva
backend-14	POST	`/reservas/{codigo}/pagos`	Registrar pago en una reserva

Tabla 3: Endpoints documentados.

Además de los endpoints mínimos, las evidencias de la primera parte usan `POST /reservas/{codigo}/cancelar` y los tres endpoints de reportes (`/reportes/ventas-por-mes`, `/reportes/destinos-mas-vendidos` y `/reportes/ocupacion-vuelos`).

4. Evidencias de las reglas (primera parte)

4.1. Evidencia 1. El validador de esquema rechaza un documento inválido

Descripción. Las colecciones `vuelos`, `hoteles` y `reservas` tienen un validador `$jsonSchema` con `validationLevel: strict` y `validationAction: error`. La prueba se hizo directamente en mongosh desde MongoDB Compass, sin pasar por la API: se intentó insertar en `vuelos` un documento con origen `"Bogotá"` en lugar del código IATA de tres letras (`"BOG"`).

Resultado de la prueba. MongoDB rechaza la inserción con `MongoServerError: Document failed validation` y el conteo de documentos con el código de prueba es 0 antes y después del intento: el documento inválido no quedó guardado. La figura 1 muestra el validador de la colección `vuelos` (campos obligatorios, `pattern ^[A-Z]{3}$` en origen y destino, entre 1 y 3 tarifas con nombre Basica, Clasica o Flexible) y la figura 2 muestra el rechazo del `insertOne`.

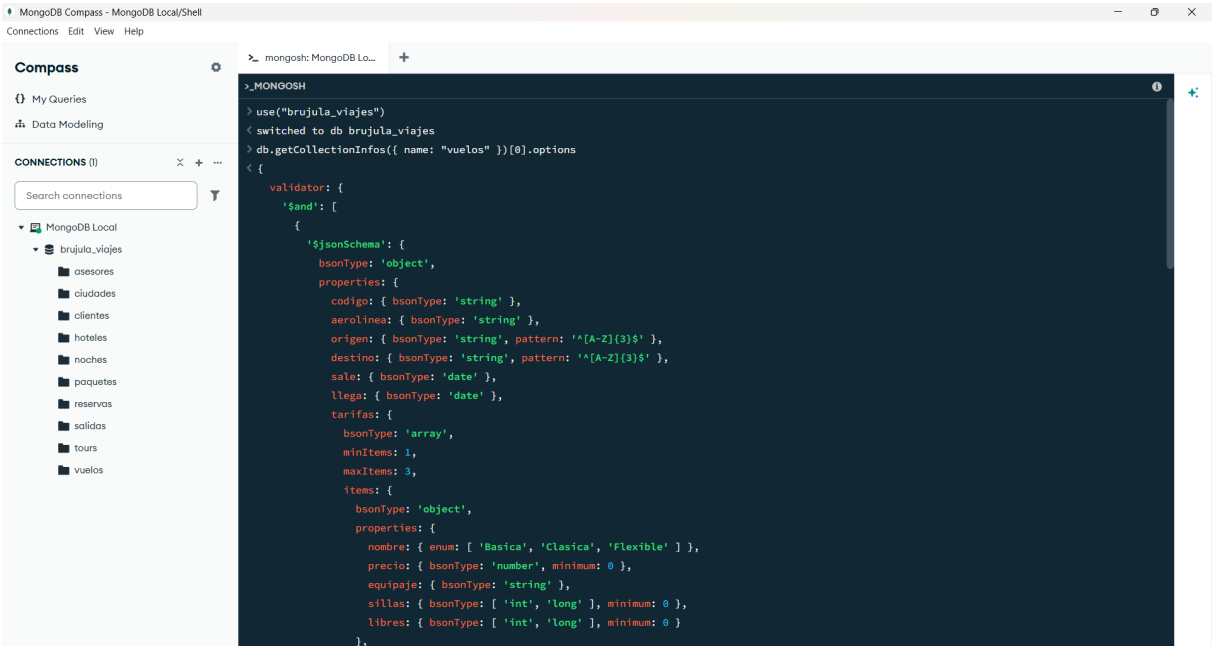


Figura 1: Evidencia 1 en mongosh (Compass): validador `$jsonSchema` de la colección `vuelos`.

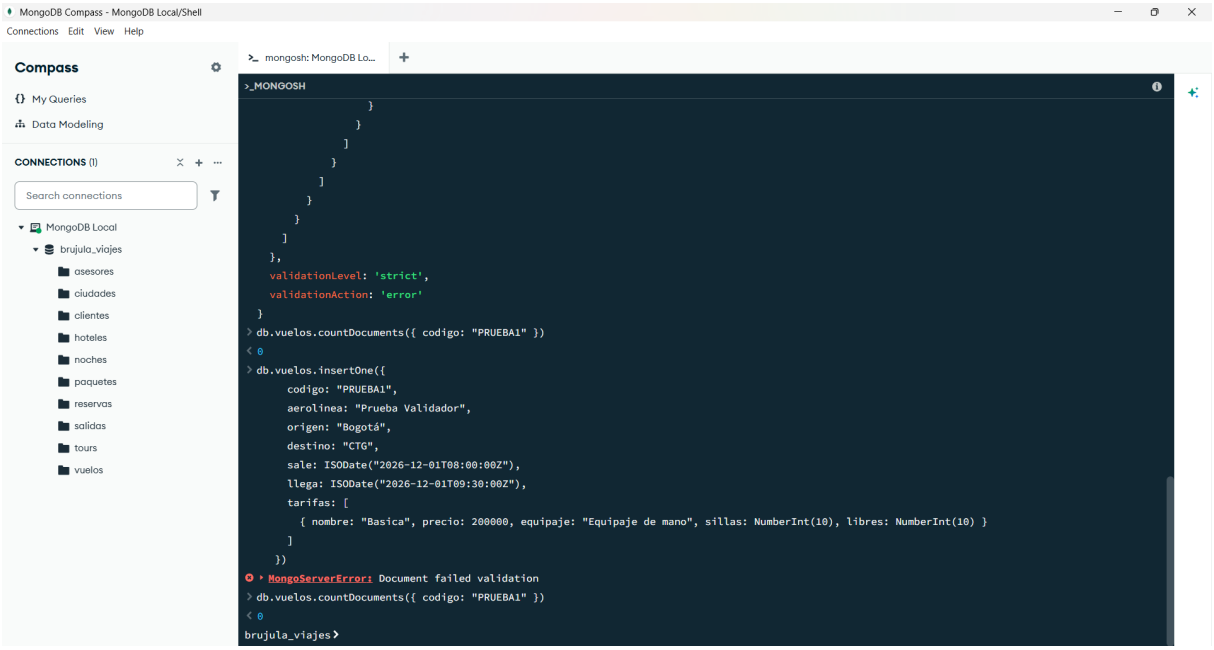


Figura 2: Evidencia 1 en mongosh (Compass): el `insertOne` con origen `"Bogotá"` falla con `Document failed validation` y el conteo sigue en 0.

4.2. Evidencia 2. Reserva rechazada por falta de cupo (409)

Descripción. Se intentó crear una reserva de 3 sillas en tarifa Basica del vuelo WG7310 (Wingo, MDE→ADZ), que solo tiene 2 sillas libres en esa tarifa. La escritura condicional que descuenta el inventario solo se aplica si hay sillas suficientes; el comportamiento esperado cuando no se cumple la condición es que la API responda 409 y no cree nada.

Resultado de la prueba. 409 Conflict con el mensaje "`Vuelo sin 3 sillas libres en tarifa Basica`". La API rechazó la solicitud y las consultas anterior y posterior del vuelo muestran el inventario sin cambios (`libres: 2`). La consulta posterior de reservas del cliente de prueba muestra RV-261105-4A4B; no se dispone de una comparación completa del listado antes y después.

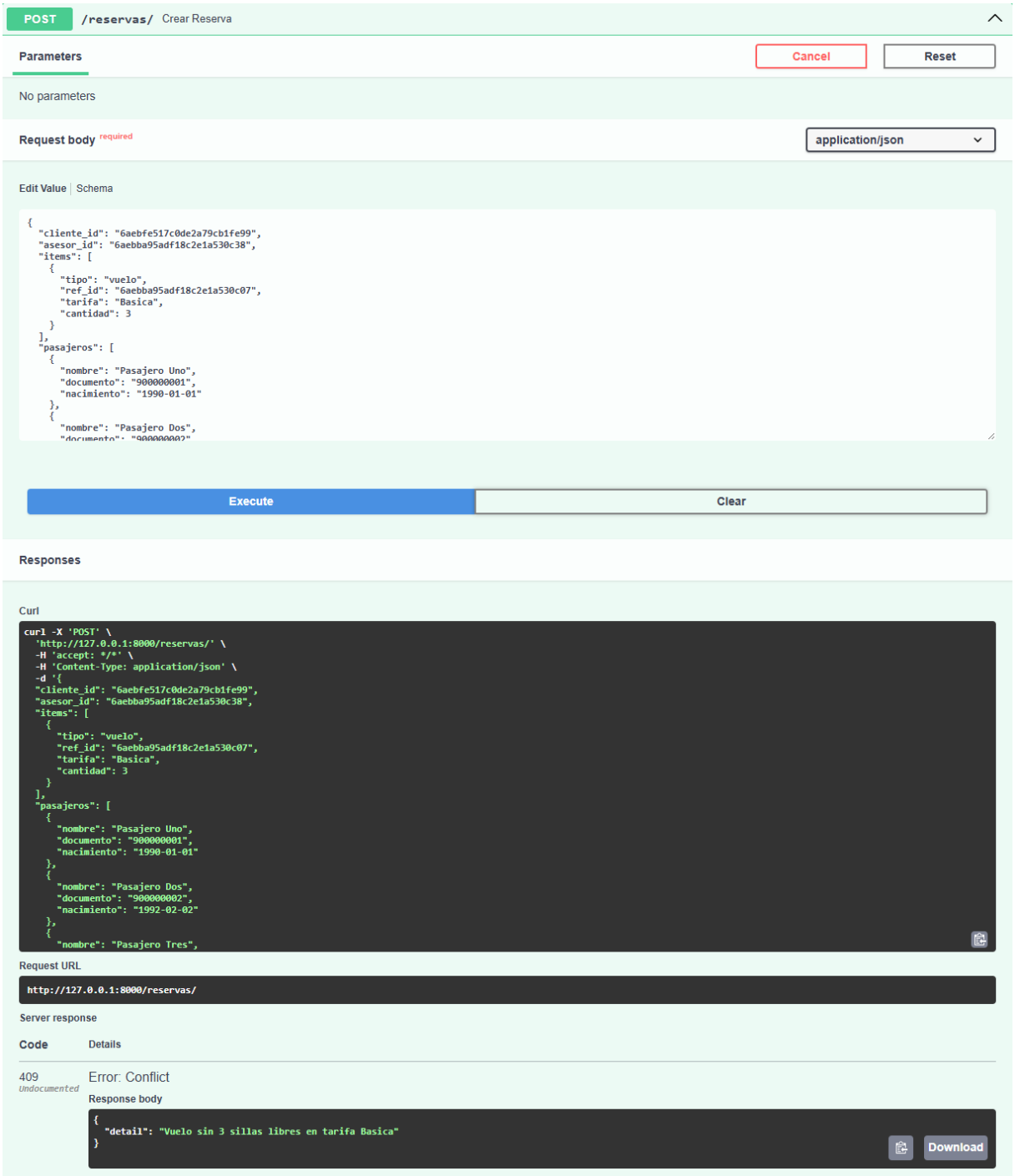


Figura 3: Evidencia 2 en Swagger: `POST /reservas/` con 3 pasajeros sobre el vuelo WG7310.

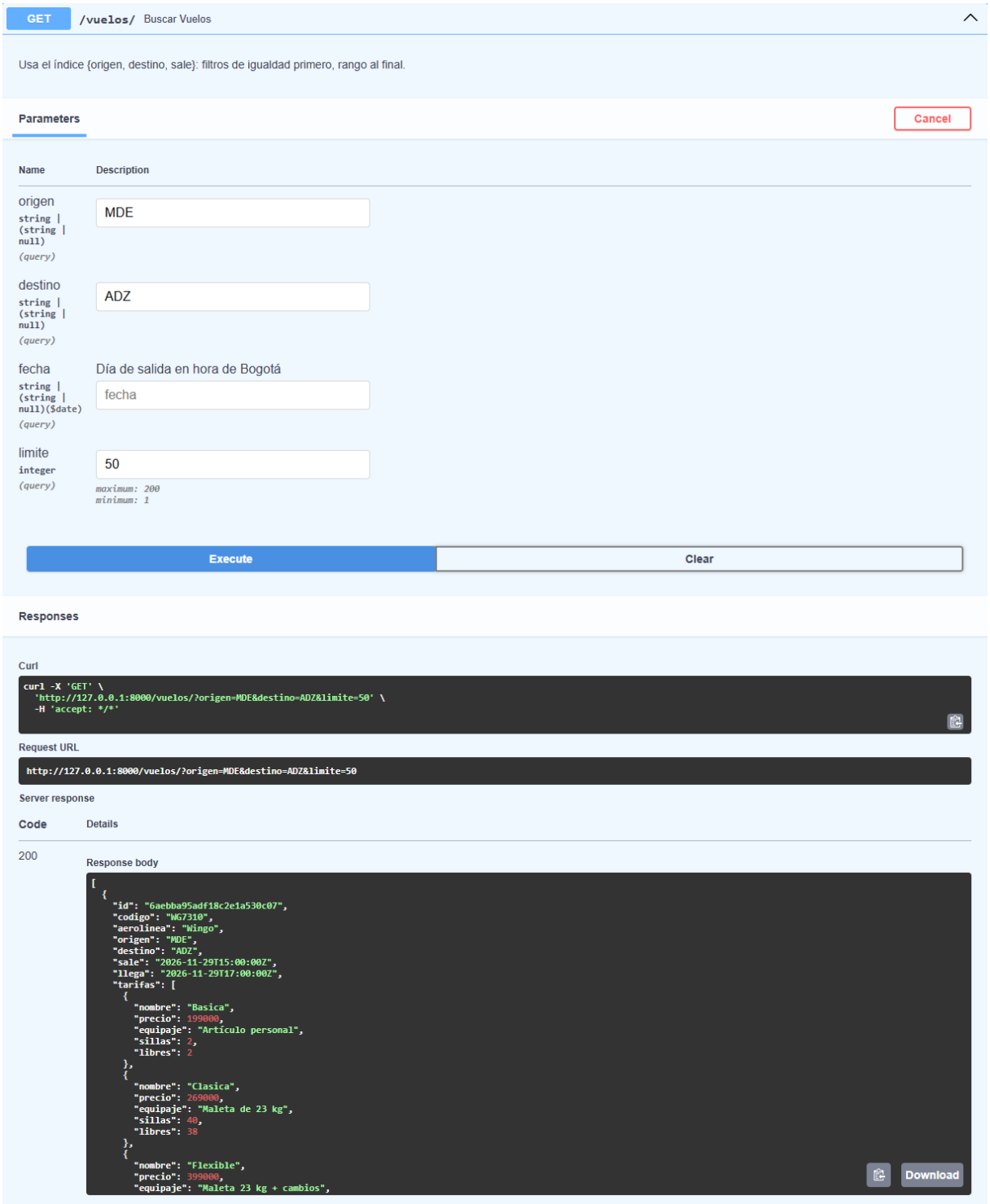


Figura 4: Evidencia 2: inventario antes del intento; el vuelo WG7310 tiene `libres: 2` en tarifa Basica.

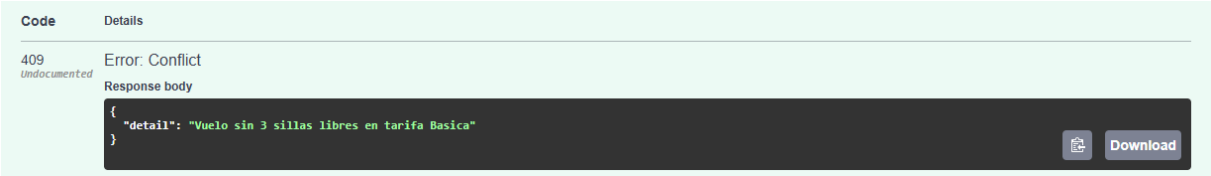


Figura 5: Evidencia 2: respuesta 409 ‘‘`Vuelo sin 3 sillas libres en tarifa Basica`’’.

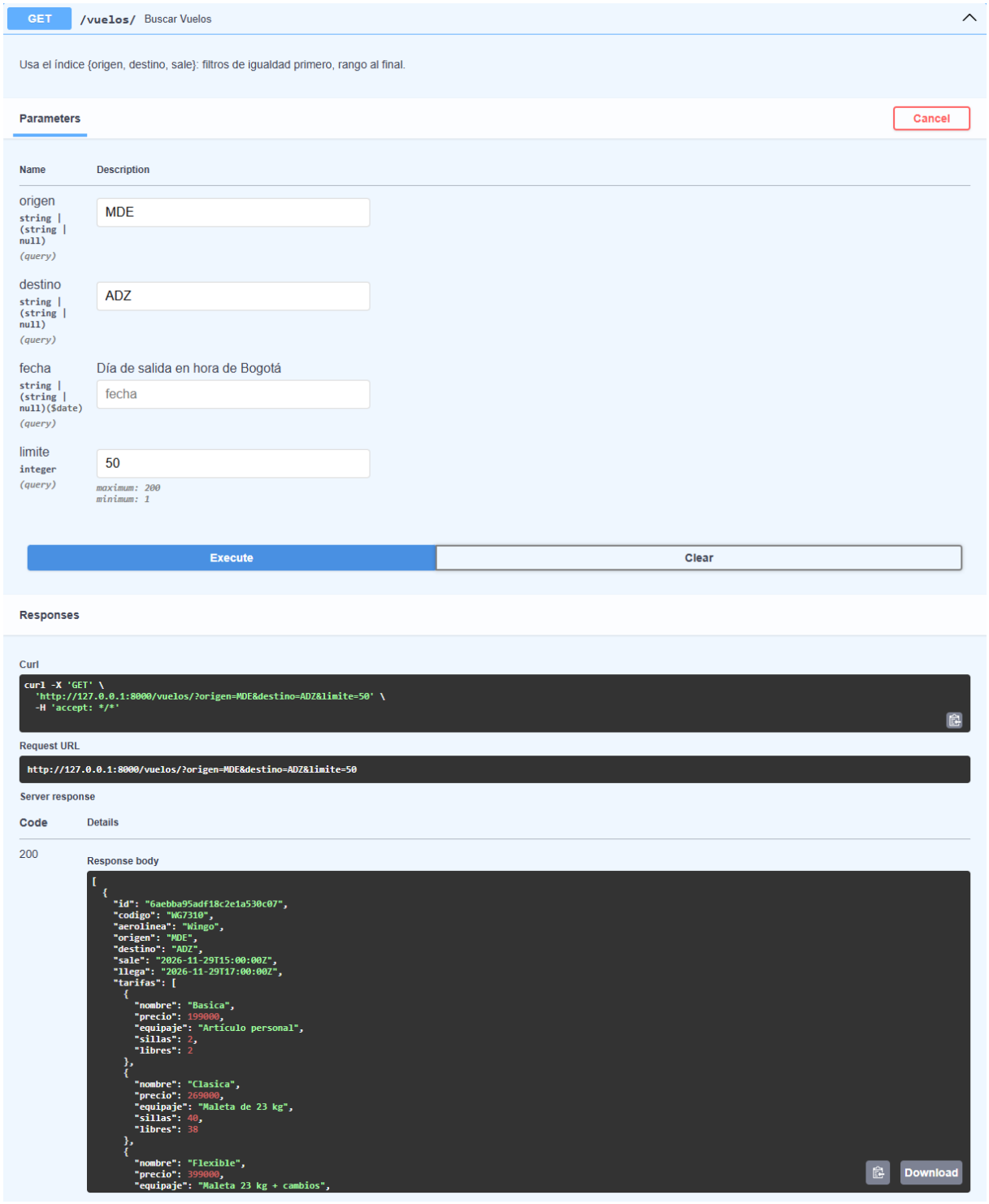


Figura 6: Evidencia 2: inventario después del intento; las sillas libres siguen en 2.

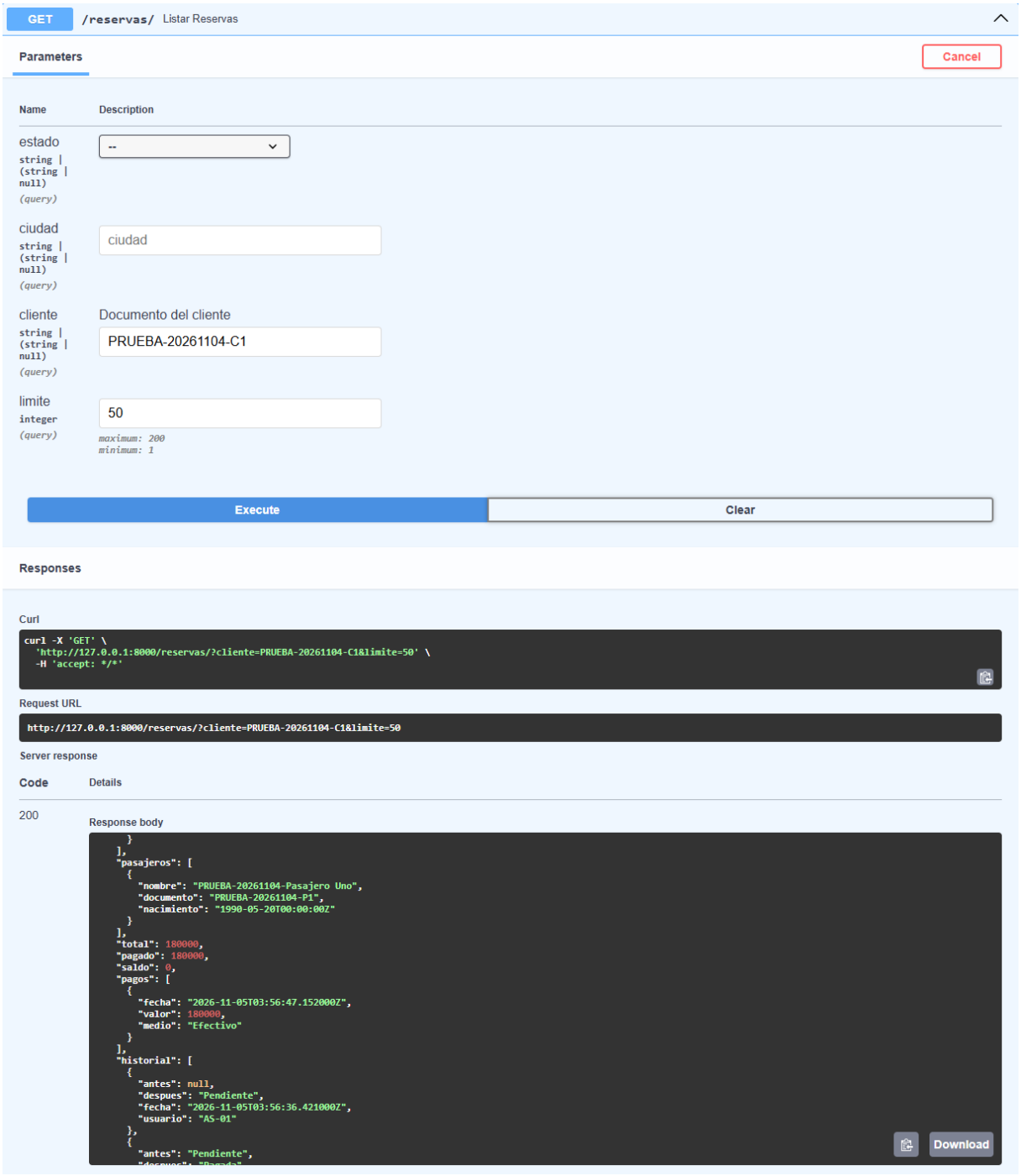


Figura 7: Evidencia 2: consulta posterior de las reservas del cliente de prueba; aparece la reserva RV-261105-4A4B.

4.3. Evidencia 3. Reserva cancelada que devuelve su inventario

Descripción. Se canceló la reserva RV-261105-4A4B (1 silla en tarifa Basica del vuelo de prueba PRUEBA-20261104-V1, BOG→CTG) con `POST /reservas/{codigo}/cancelar`, indicando el usuario AS-01. La cancelación cambia el estado y devuelve el inventario; los pagos quedan registrados.

Resultado de la prueba. 200 OK con la reserva en estado Cancelada. Las sillas libres del vuelo pasan de 9 a 10 (figuras 12 y 13). El historial conserva la traza completa:

creación (Pendiente), pago (Pagada) y cancelación (Cancelada), cada una con fecha y usuario. Una segunda cancelación sobre la misma reserva responde 409 ‘‘`La reserva ya estaba cancelada`’’ y el inventario no cambia (figuras 14 y 15): cancelar dos veces no devuelve el inventario dos veces.

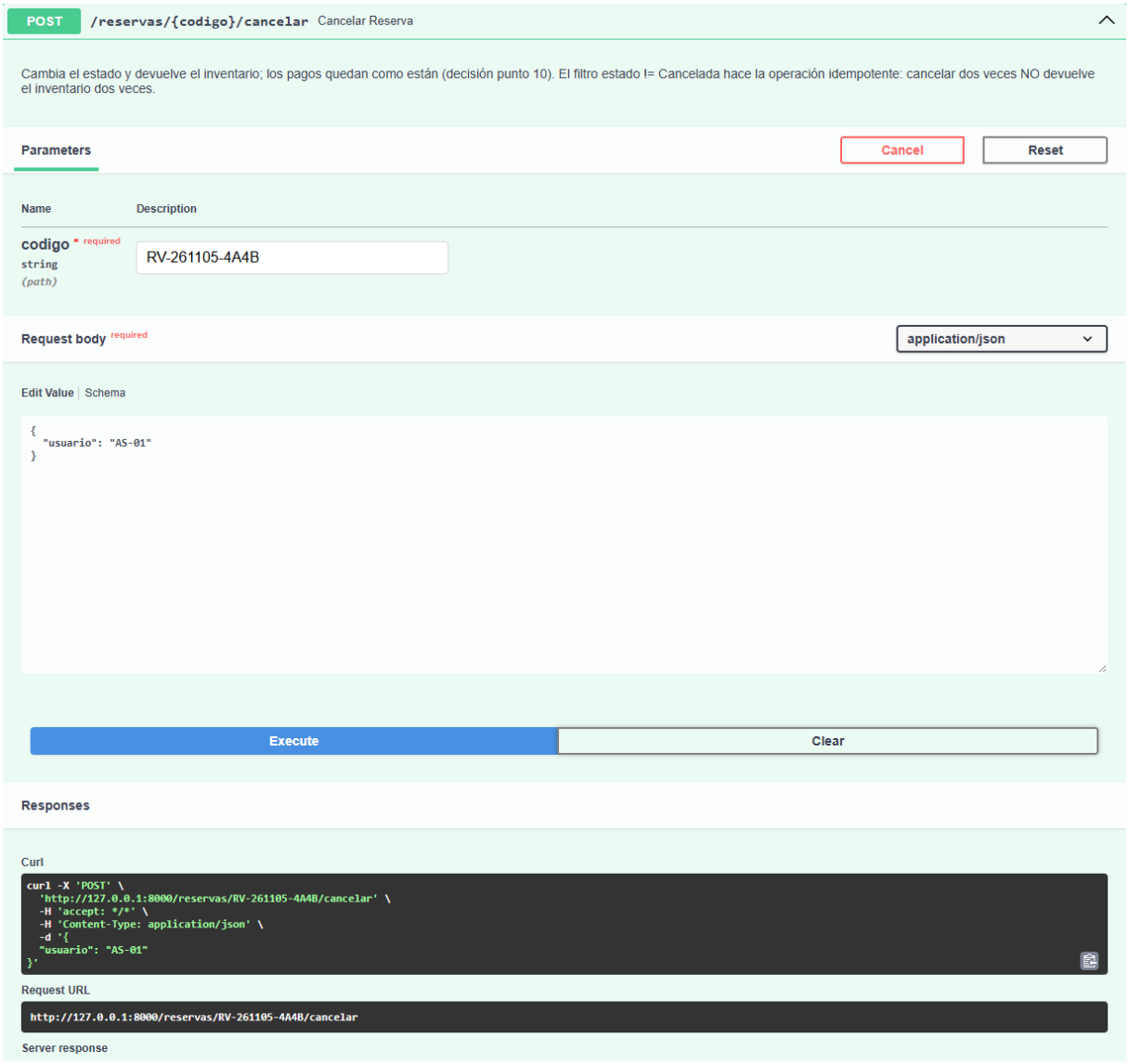


Figura 8: Evidencia 3 en Swagger: `POST /reservas/RV-261105-4A4B/cancelar` con usuario AS-01.

Code Details

200

Response body


```
{
  "id": "6aebfe747c0de2a79cb1fe9e",
  "codigo": "RV-261105-4A4B",
  "cliente_id": "6aebfe517c0de2a79cb1fe99",
  "cliente": {
    "documento": "PRUEBA-20261104-C1",
    "nombre": "PRUEBA-20261104-Cliente Uno"
  },
  "asesor_id": "6aebba95adf18c2e1a530c38",
  "asesor": "Carlos Perez",
  "creada": "2026-11-05T03:56:36.421000Z",
  "estado": "Cancelada",
  "items": [
    {
      "tipo": "vuelo",
      "ref_id": "6aebfe5a7c0de2a79cb1fe9a",
      "detalle": {
        "codigo": "PRUEBA-20261104-V1",
        "tarifa": "Basica"
      },
      "nombre": "BOG-CTG PRUEBA-20261104-V1",
      "ciudad": "CTG",
      "inicio": "2027-01-20T12:00:00Z",
      "fin": "2027-01-20T13:30:00Z",
      "cantidad": 1,
      "precio": 180000,
      "subtotal": 180000
    }
```


Download

Figura 9: Evidencia 3: respuesta 200 con la reserva en estado Cancelada.

Code Details

200

Response body


```
      "precio": 180000,
      "subtotal": 180000
    }
  ],
  "pasajeros": [
    {
      "nombre": "PRUEBA-20261104-Pasajero Uno",
      "documento": "PRUEBA-20261104-P1",
      "nacimiento": "1990-05-20T00:00:00Z"
    }
  ],
  "total": 180000,
  "pagado": 180000,
  "saldo": 0,
  "pagos": [
    {
      "fecha": "2026-11-05T03:56:47.152000Z",
      "valor": 180000,
      "medio": "Efectivo"
    }
  ],
  "historial": [
    {
      "antes": null,
      "despues": "Pendiente",
      "fecha": "2026-11-05T03:56:36.421000Z",
      "usuario": "AS-01"
    },
    {
```


Download

Figura 10: Evidencia 3: el pago registrado queda en la reserva; el total y lo pagado no cambian.

Code Details

200

Response body


```
  "pagos": [
    {
      "fecha": "2026-11-05T03:56:47.152000Z",
      "valor": 180000,
      "medio": "Efectivo"
    }
  ],
  "historial": [
    {
      "antes": null,
      "despues": "Pendiente",
      "fecha": "2026-11-05T03:56:36.421000Z",
      "usuario": "AS-01"
    },
    {
      "antes": "Pendiente",
      "despues": "Pagada",
      "fecha": "2026-11-05T03:56:47.152000Z",
      "usuario": "AS-01"
    },
    {
      "antes": "Pagada",
      "despues": "Cancelada",
      "fecha": "2026-11-05T03:56:52.532000Z",
      "usuario": "AS-01"
    }
  ]
}
```


Download

Figura 11: Evidencia 3: historial con Pendiente → Pagada → Cancelada, cada cambio con fecha y usuario.

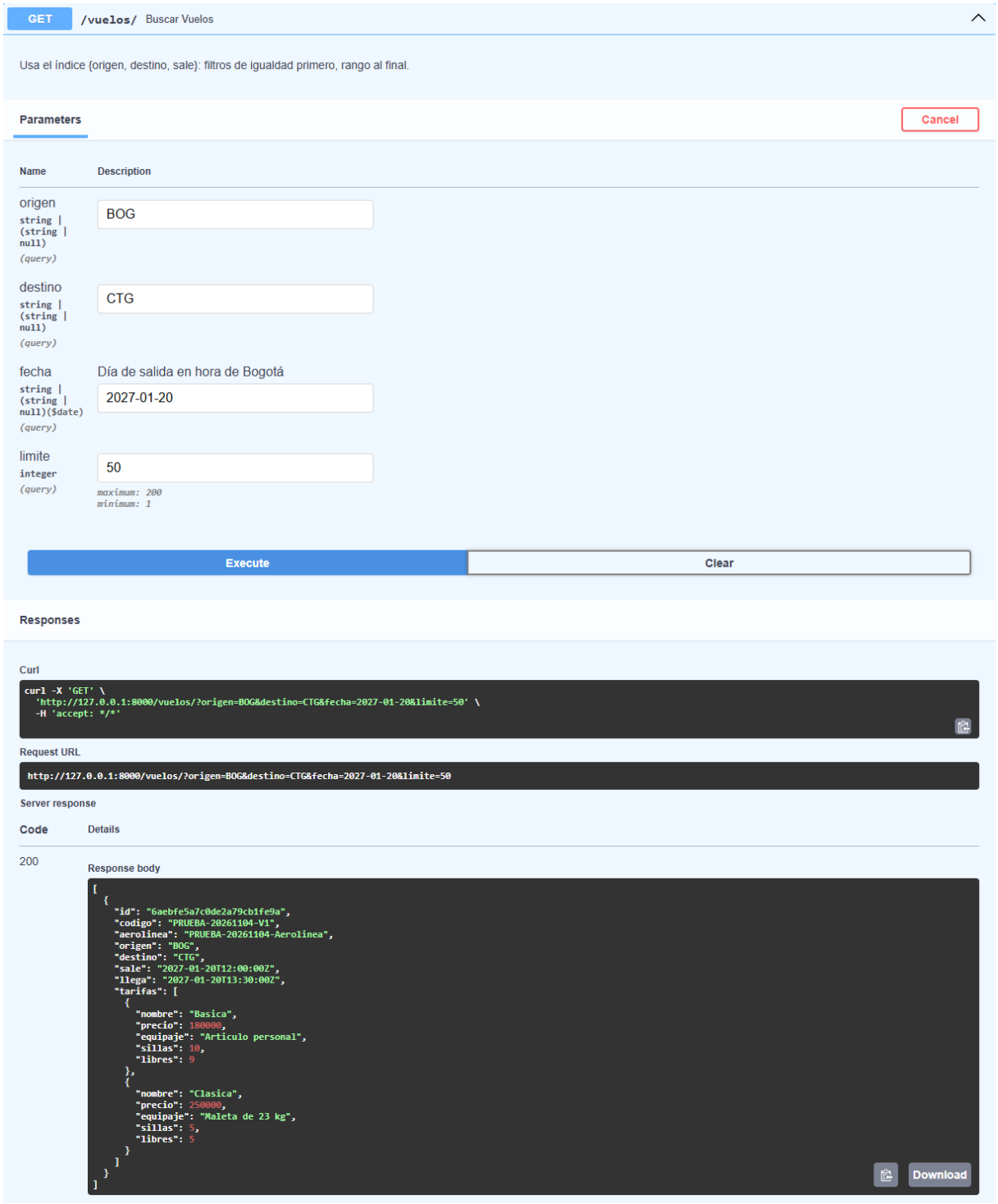


Figura 12: Evidencia 3: inventario antes de cancelar; el vuelo de prueba tiene `libres: 9` en tarifa Basica.

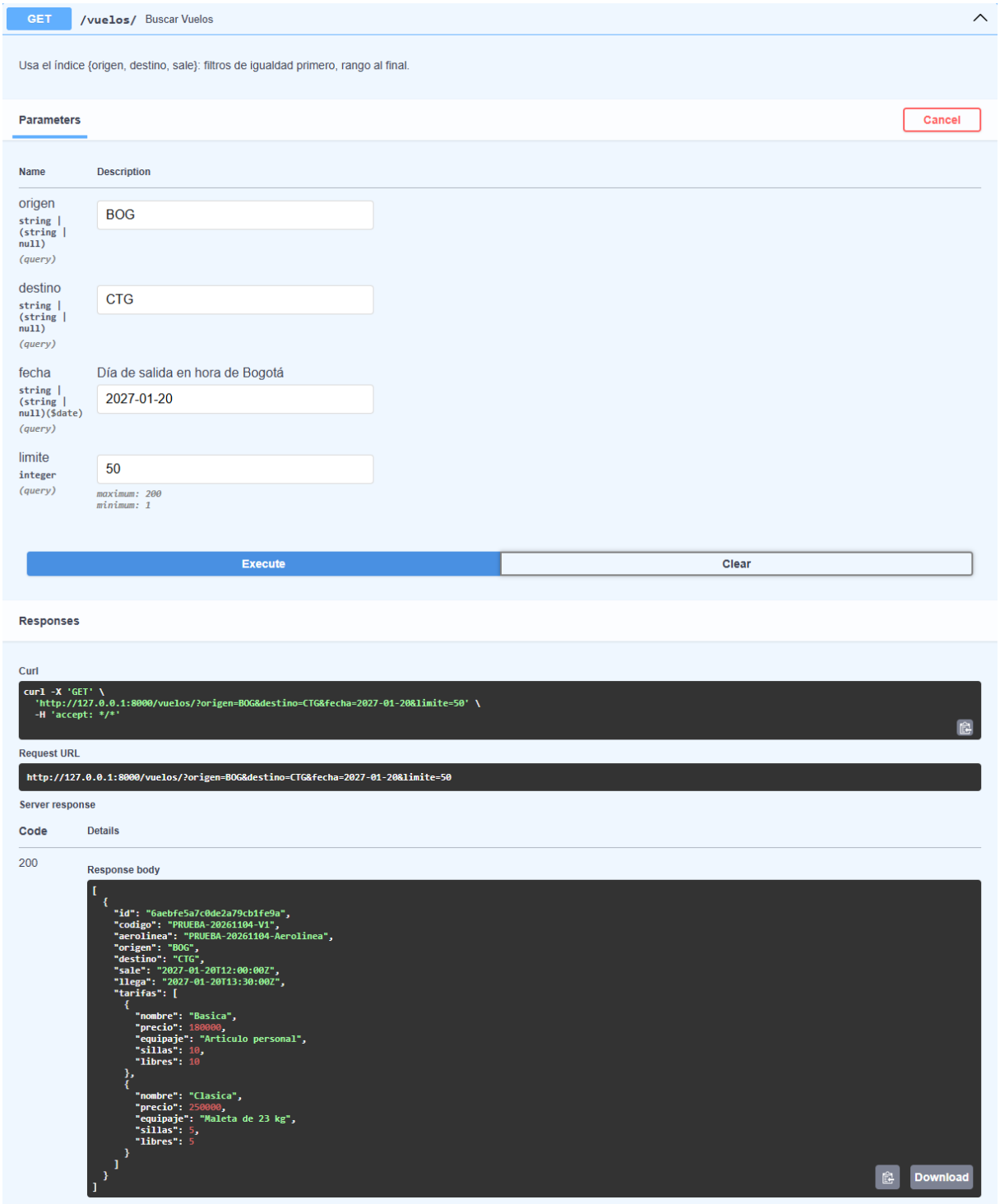


Figura 13: Evidencia 3: inventario después de cancelar; `libres` sube a 10.

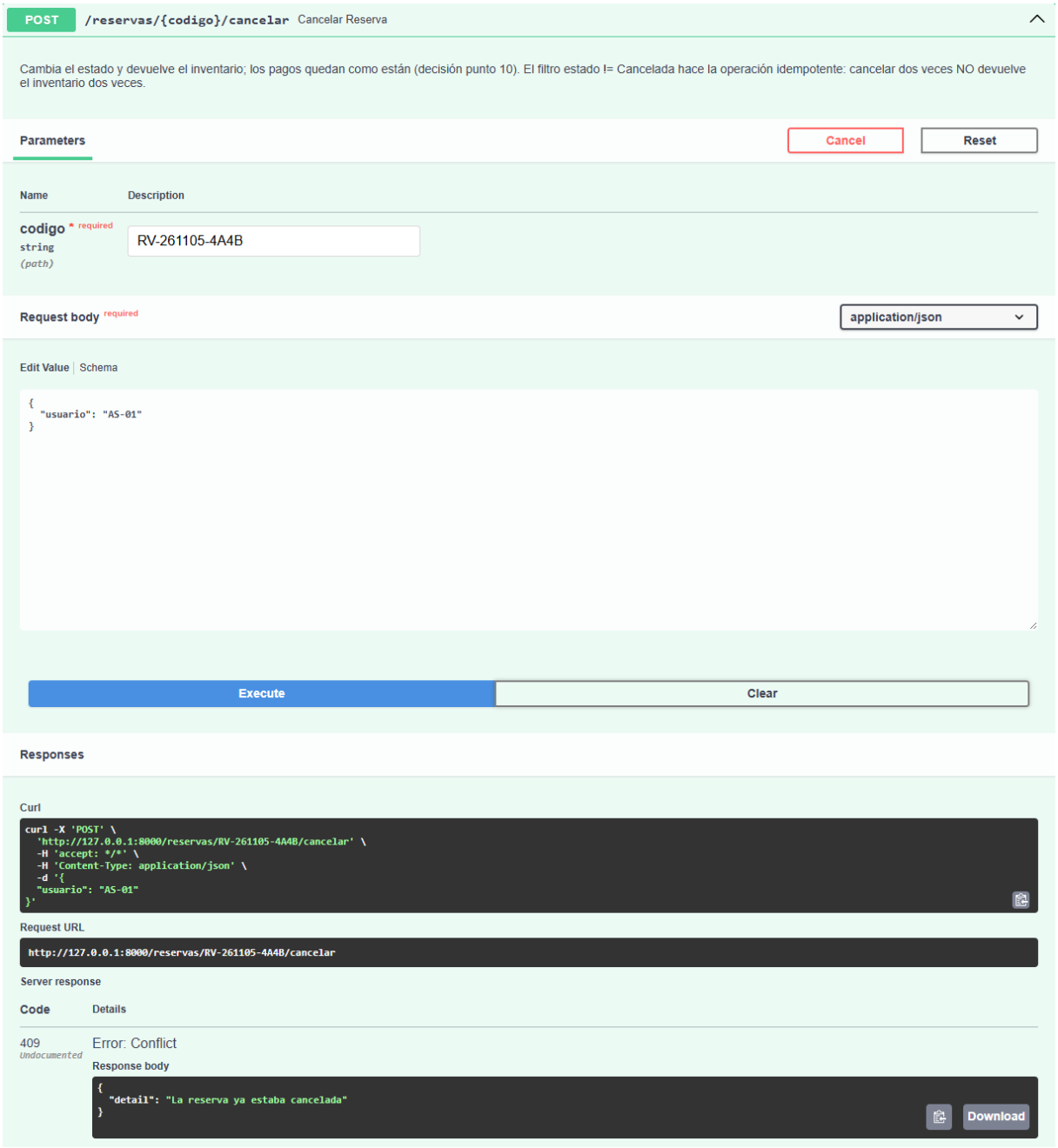


Figura 14: Evidencia 3: la segunda cancelación responde 409 "`La reserva ya estaba cancelada`".

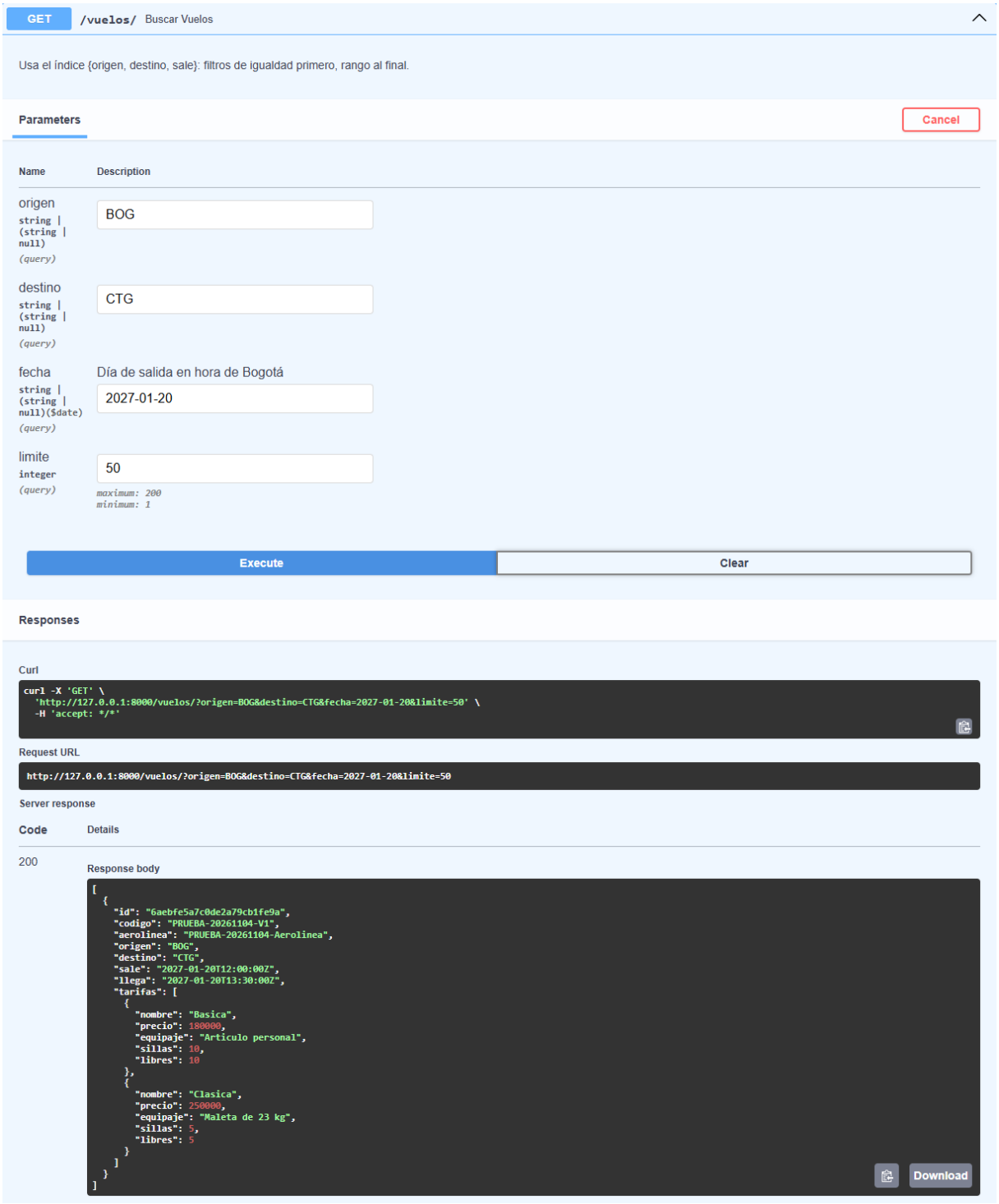


Figura 15: Evidencia 3: inventario después de la segunda cancelación; `libres` sigue en 10.

4.4. Evidencia 4. Reporte B1: ventas por mes

Descripción. `GET /reportes/ventas-por-mes` resuelve el reporte B1 con un pipeline de agregación: total vendido y número de reservas por mes, sin contar las canceladas.

Resultado de la prueba. 200 OK con los totales por mes: septiembre de 2026 ($16.060.000 en 7 reservas), octubre ($18.819.600 en 6) y noviembre ($14.320.800 en 5).

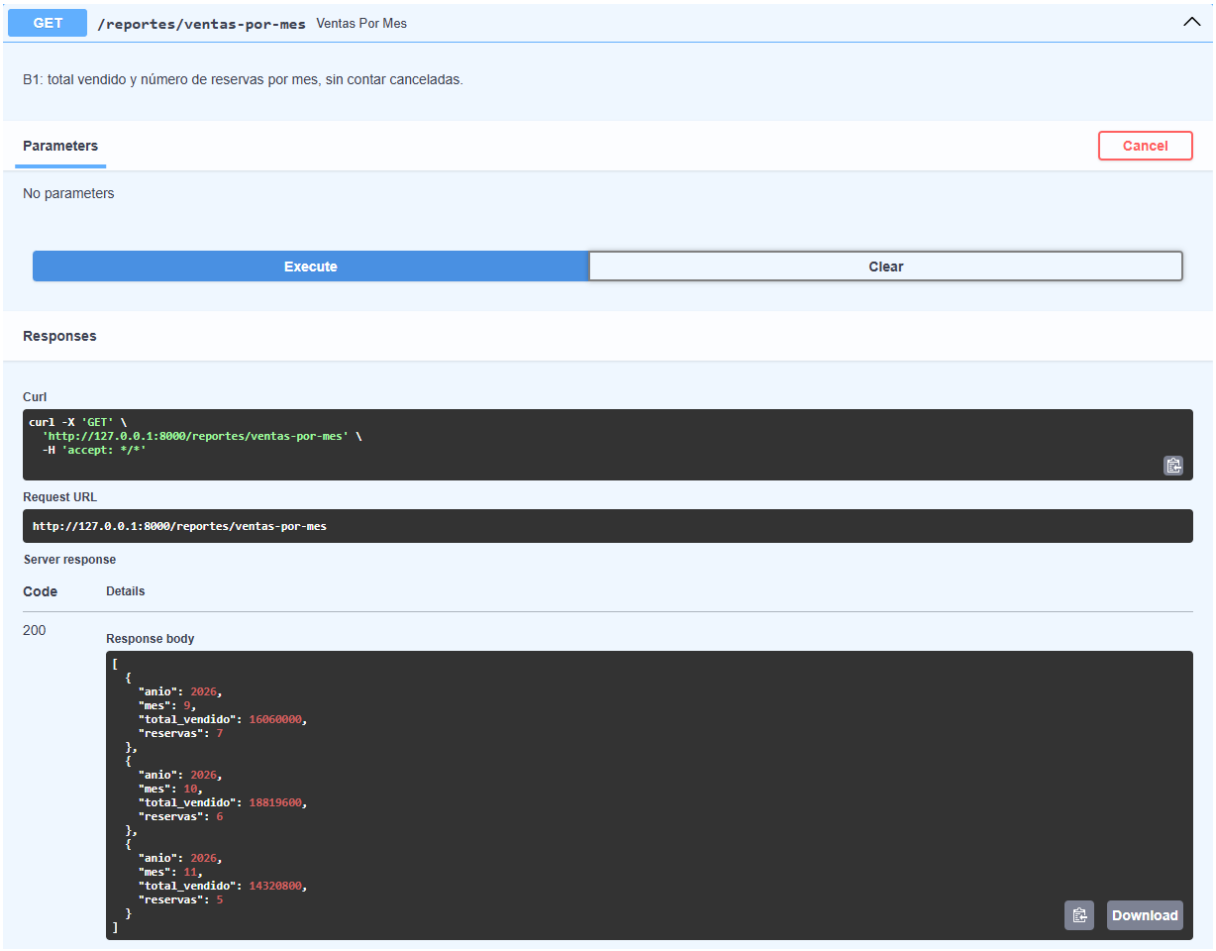


Figura 16: Evidencia 4 en Swagger: reporte B1 de ventas por mes.

4.5. Evidencia 5. Reporte B2: destinos más vendidos

Descripción. `GET /reportes/destinos-mas-vendidos` resuelve el reporte B2 con un pipeline de agregación: las 5 ciudades de destino con más ingresos, con su número de pasajeros, sin contar las canceladas.

Resultado de la prueba. 200 OK con el top 5 ordenado por ingresos: CTG ($21.610.000, 12 pasajeros), SMR ($12.890.000, 3), MDE ($9.822.400, 9), ADZ ($2.698.000, 2) y BOG ($1.360.000, 5).

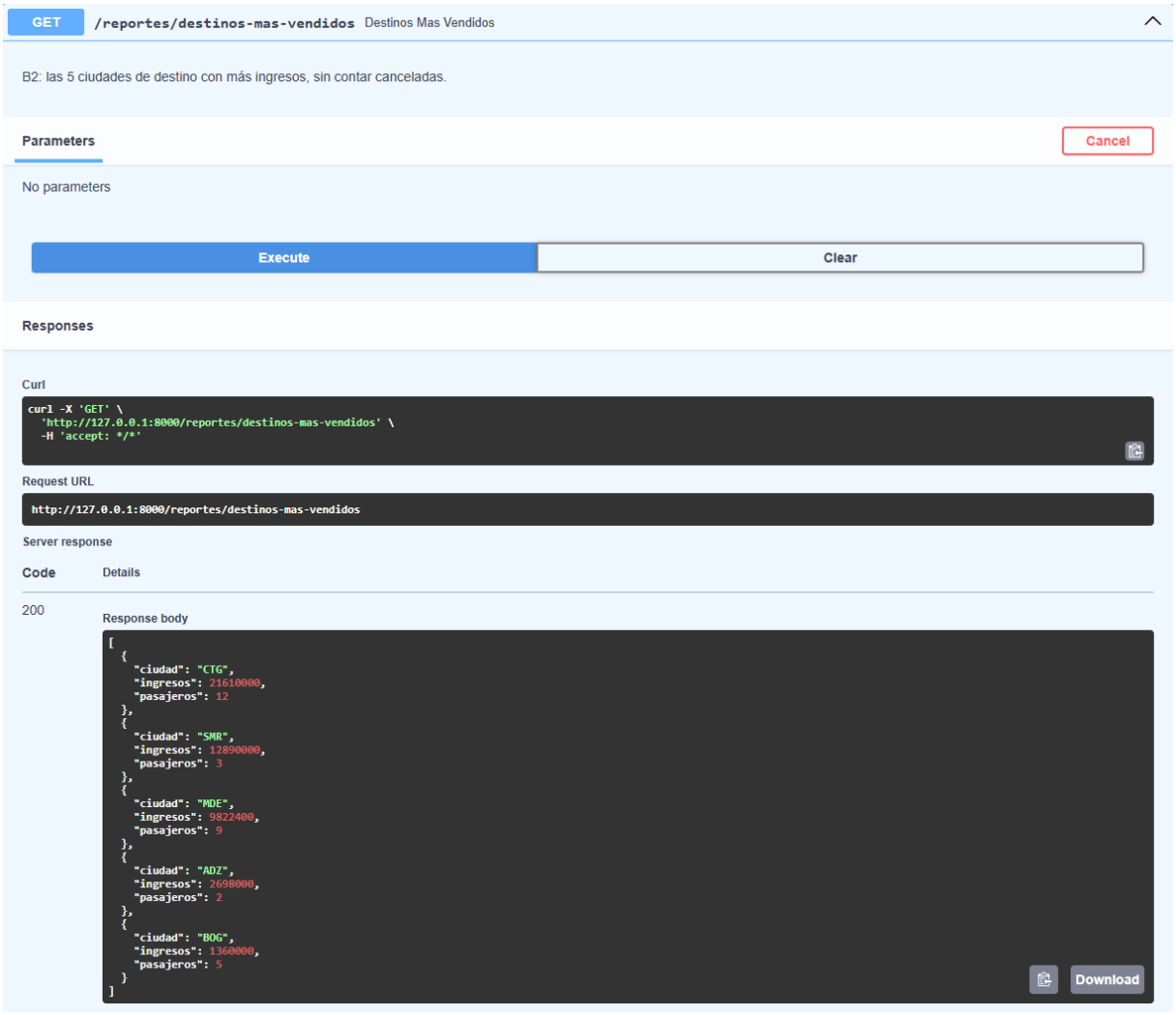


Figura 17: Evidencia 5 en Swagger: reporte B2 de destinos más vendidos.

4.6. Evidencia 6. Reporte B3: ocupación de vuelos

Descripción. `GET /reportes/ocupacion-vuelos` resuelve el reporte B3 con un pipeline de agregación: por vuelo, sillas totales, vendidas y porcentaje de ocupación, de mayor a menor. Sale del inventario, que ya excluye las canceladas porque cancelar devuelve las sillas a `libres`.

Resultado de la prueba. 200 OK con la ocupación por vuelo. El vuelo de prueba PRUEBA-20261104-V1 aparece con 15 sillas totales, 0 vendidas y 0 % de ocupación, coherente con la cancelación de la evidencia 3.

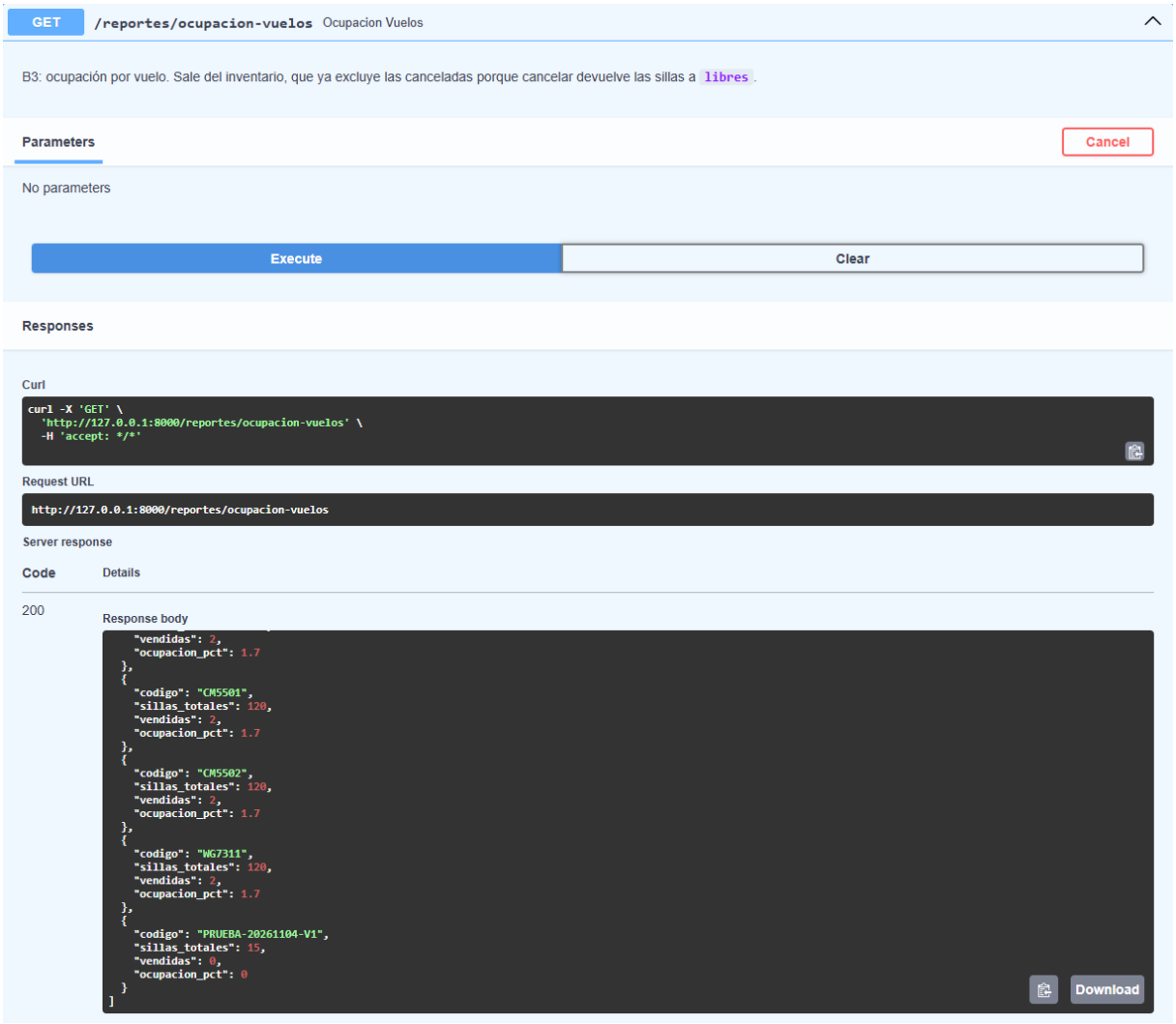


Figura 18: Evidencia 6 en Swagger: reporte B3 de ocupación de vuelos.

5. Pruebas de los endpoints (segunda parte)

5.1. Clientes

Un cliente se registra con documento único, nombre, correo, teléfono y, opcionalmente, sus preferencias de viaje.

5.1.1. backend-01. Listar clientes

Método	GET
Ruta	`/clientes/`
Colecciones	`clientes`
Reglas en la base	Ninguna

Descripción. Devuelve la lista de clientes registrados con su documento, nombre, correo y teléfono.

Resultado de la prueba. 200 OK con la lista de clientes de los datos de prueba.

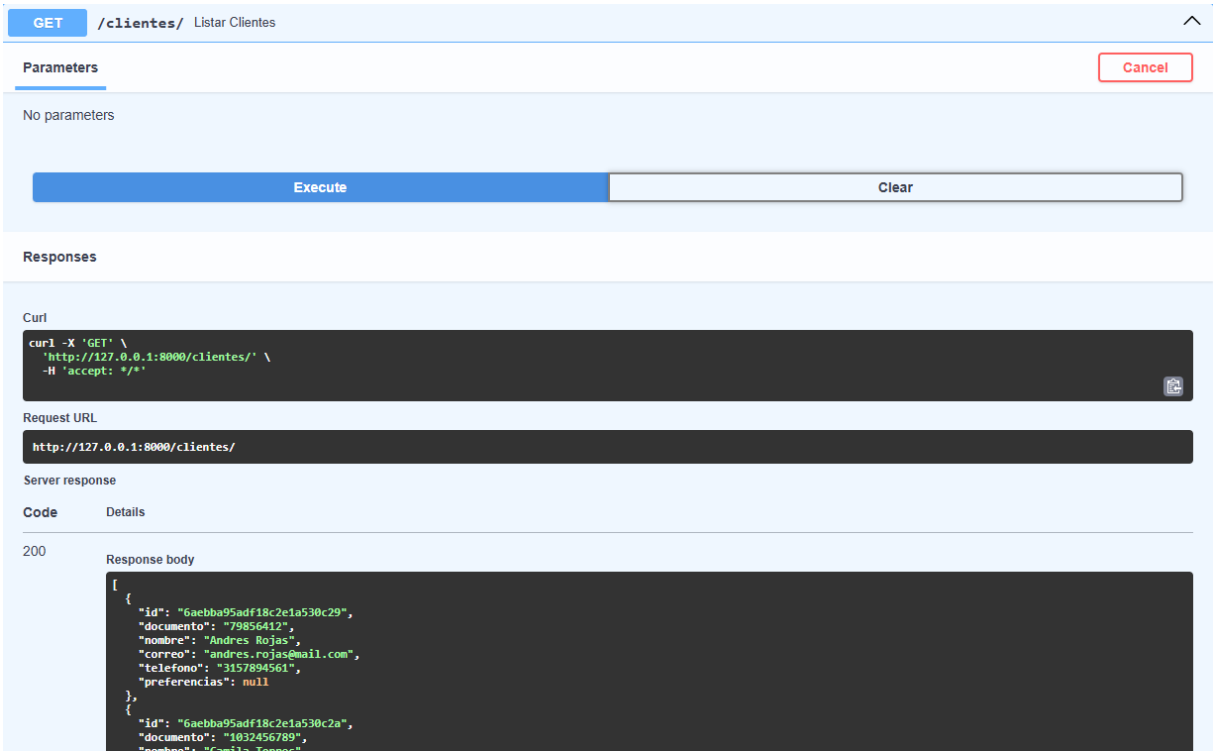


Figura 19: Prueba backend-01 en Swagger: listar clientes.

5.1.2. backend-02. Registrar cliente

Método	POST
Ruta	`/clientes/`
Colecciones	`clientes`
Reglas en la base	Índice único en `documento`

Descripción. Registra un cliente nuevo. El DTO valida el documento (5 a 20 caracteres), el formato del correo y el teléfono (7 a 15 dígitos); las preferencias son opcionales. El documento no se puede repetir.

Resultado de la prueba. 201 Created; se crea el cliente PRUEBA-20261104-C1 y luego aparece en el listado de clientes.

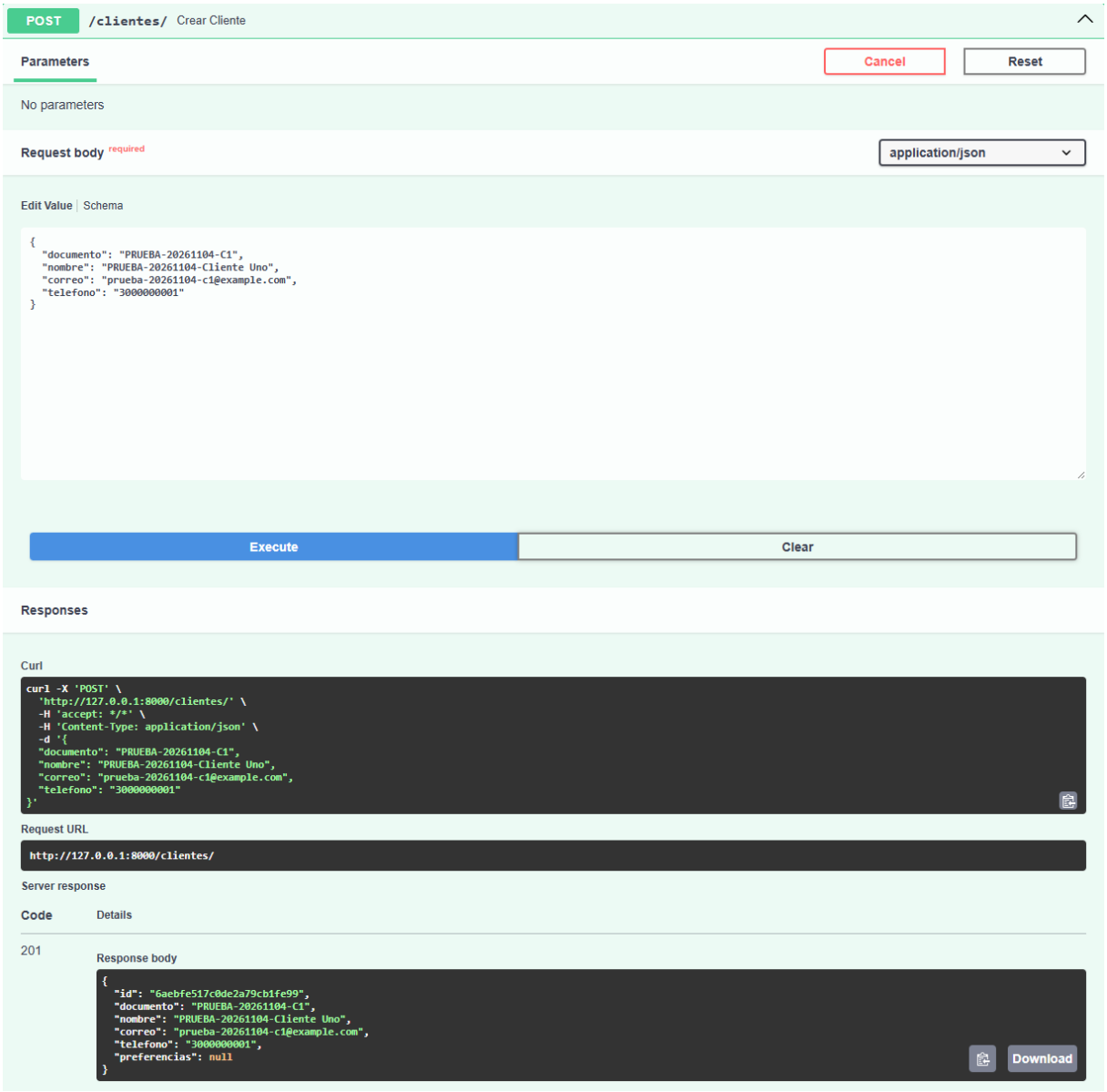


Figura 20: Prueba backend-02 en Swagger: registrar cliente.

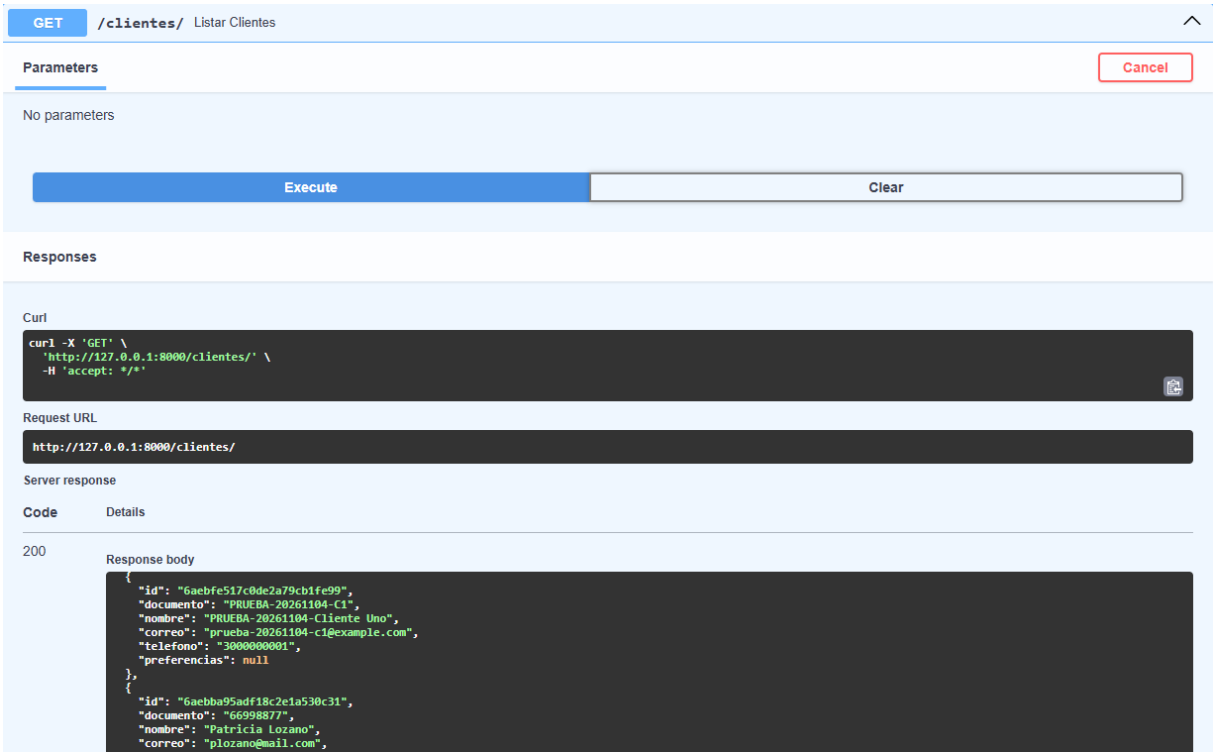


Figura 21: Prueba backend-02: el cliente creado aparece en el listado.

5.2. Vuelos

Un vuelo tiene código único, aerolínea, origen y destino con código IATA, fecha y hora de salida y llegada, y de 1 a 3 tarifas (Basica, Clasica, Flexible), cada una con precio, equipaje y sillas.

5.2.1. backend-03. Buscar vuelos

Método	GET
Ruta	`/vuelos/`
Colecciones	`vuelos`
Reglas en la base	Ninguna

Descripción. Devuelve el catálogo de vuelos con sus tarifas y sillas libres. Admite los parámetros opcionales `origen` y `destino` (IATA), `fecha` (día de salida en hora de Bogotá) y `limite`; usa el índice (origen, destino, sale).

Resultado de la prueba. 200 OK con el catálogo; el vuelo BV1234 (Avianca, BOG→CTG) muestra sus tres tarifas con precios y sillas libres. Con el filtro `origen=BOG` solo llegan los vuelos que salen de Bogotá.

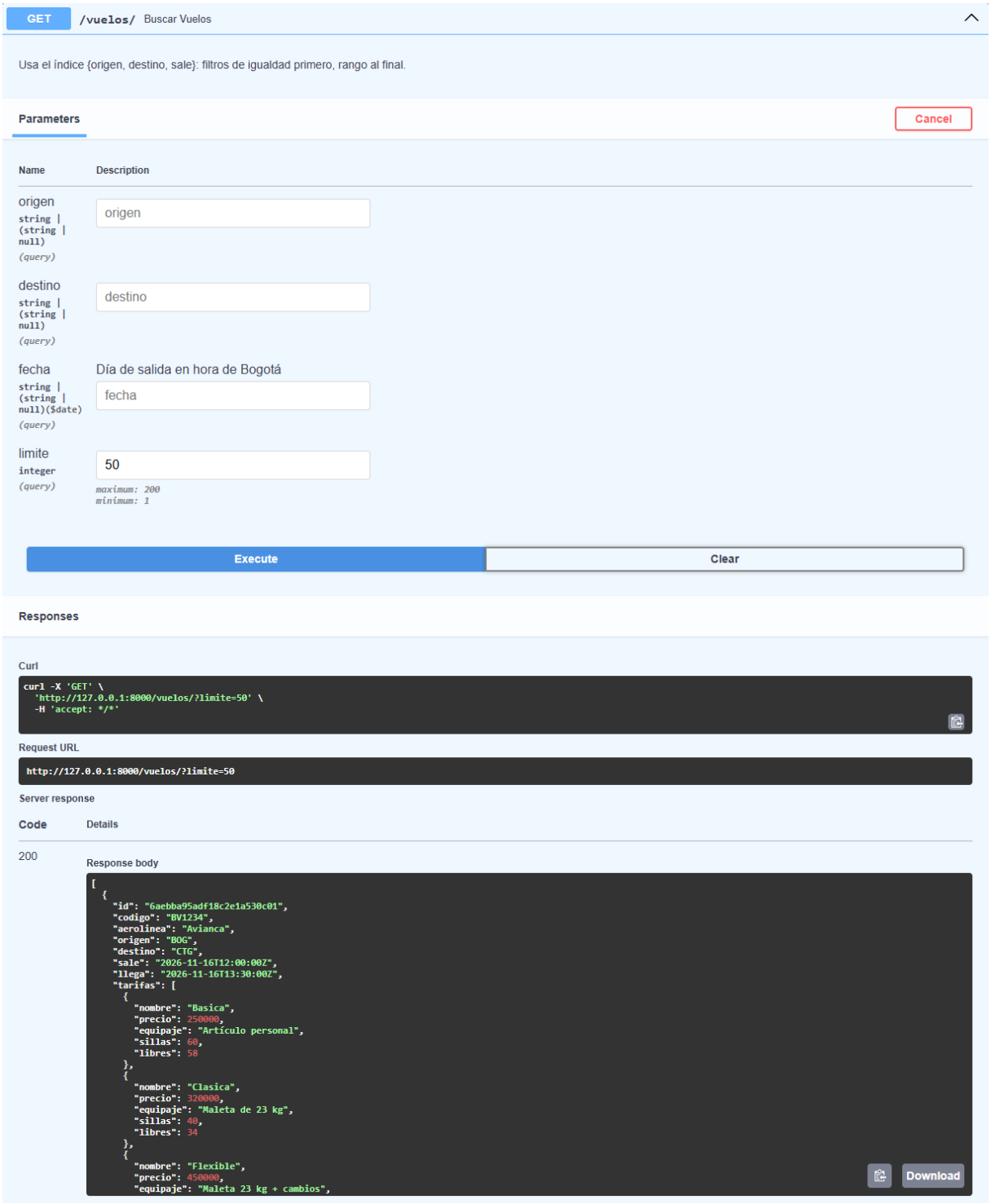


Figura 22: Prueba backend-03 en Swagger: buscar vuelos.

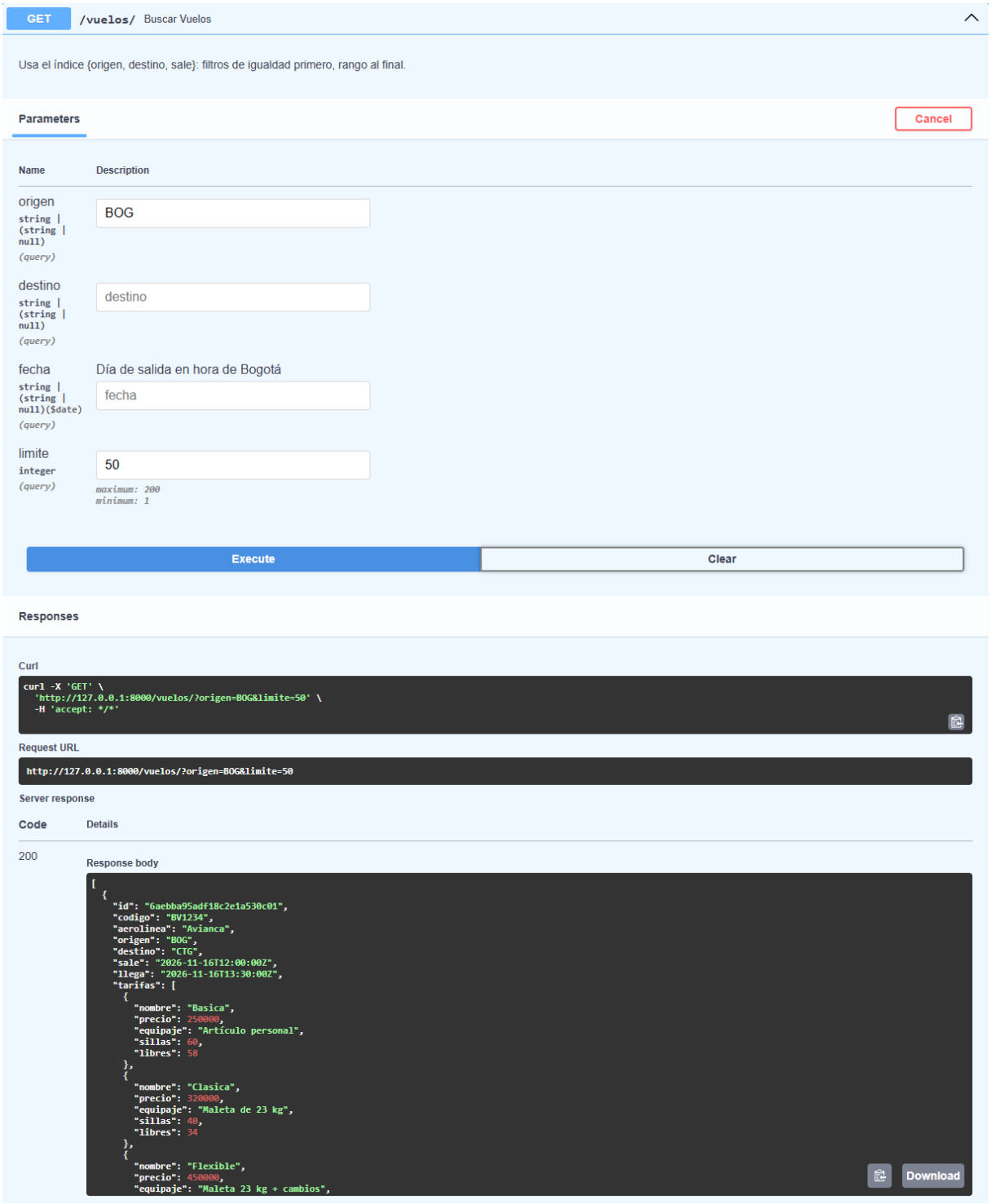


Figura 23: Prueba backend-03: búsqueda con filtro `origen=BOG`.

5.2.2. backend-04. Registrar vuelo

Método	POST
Ruta	`/vuelos/`
Colecciones	`vuelos`
Reglas en la base	Índice único en `codigo`; validador `$jsonSchema`

Descripción. Registra un vuelo con su aerolínea, origen y destino IATA distintos, horas de salida y llegada con zona horaria y de 1 a 3 tarifas sin nombre repetido. Al crearse, las sillas libres de cada tarifa arrancan iguales a las sillas totales.

Resultado de la prueba. 201 Created; se crea el vuelo PRUEBA-20261104-V1 (BOG→CTG, sale el 2027-01-20) con tarifa Basica de $180.000 (10 sillas) y Clasica de $250.000 (5 sillas), y aparece en la búsqueda con `libres` igual a `sillas`.

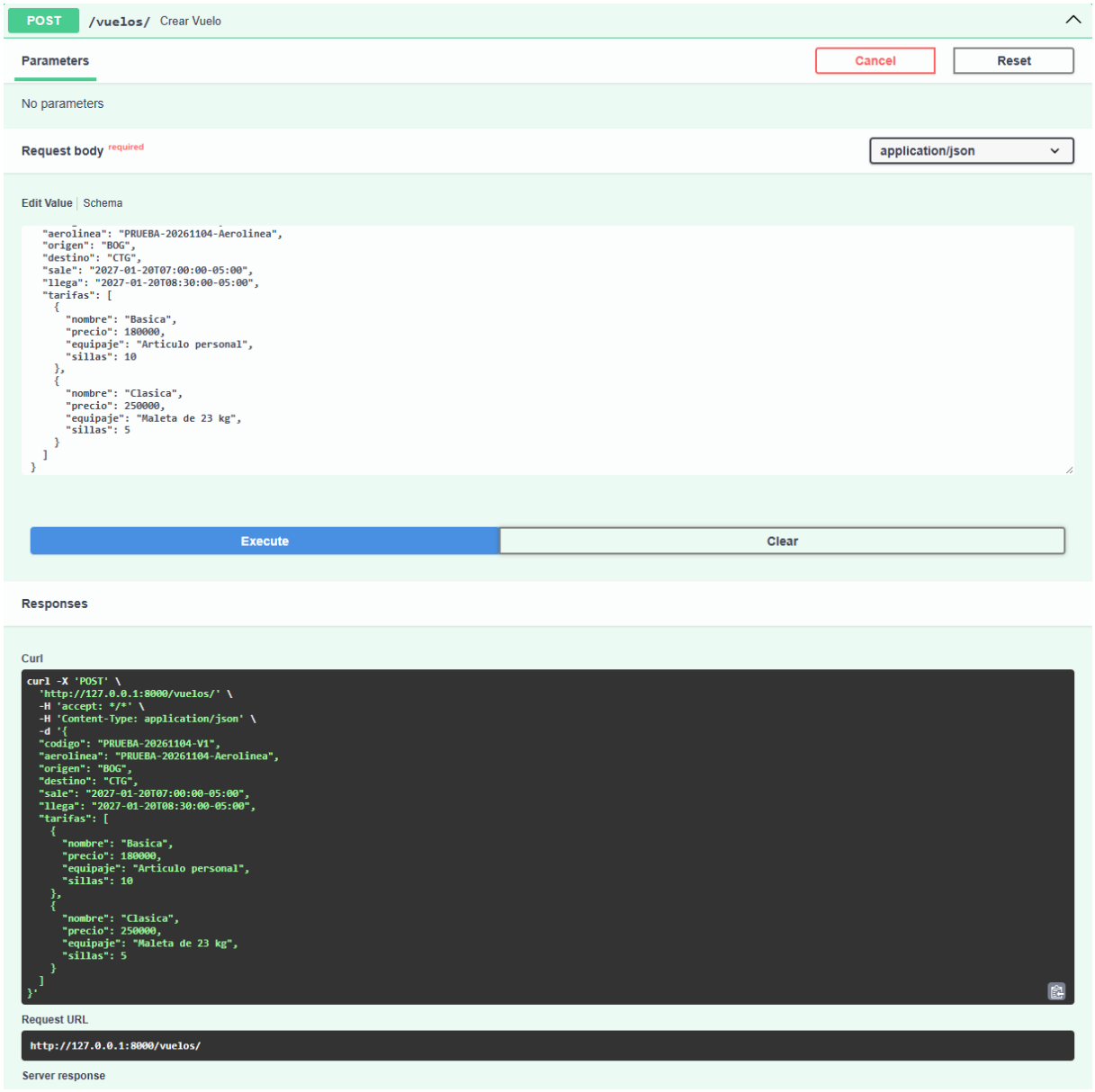


Figura 24: Prueba backend-04 en Swagger: registrar vuelo.

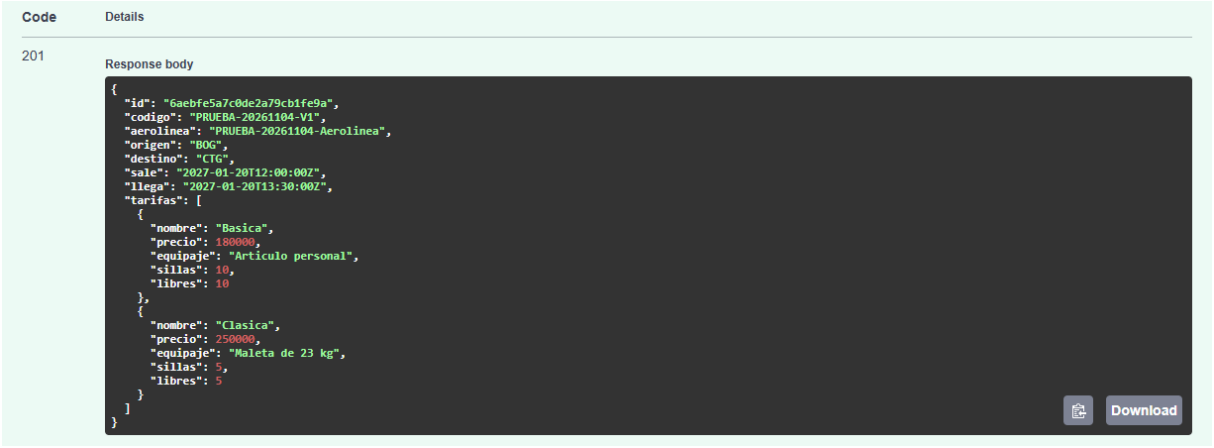


Figura 25: Prueba backend-04: respuesta 201 con el vuelo creado.



Figura 26: Prueba backend-04: el vuelo creado aparece en la búsqueda.

5.3. Hoteles

Un hotel tiene nombre, ciudad, dirección, categoría de 1 a 5 estrellas, servicios y ubicación geográfica, y ofrece de 1 a 4 tipos de habitación (Sencilla, Doble, Familiar, Suite) con capacidad, precio por noche y disponibilidad por noche.

5.3.1. backend-05. Listar hoteles

Método	GET
Ruta	`/hoteles/`
Colecciones	`hoteles`
Reglas en la base	Ninguna

Descripción. Devuelve el catálogo de hoteles con su ciudad, estrellas y habitaciones. Admite el parámetro opcional `ciudad` (IATA).

Resultado de la prueba. 200 OK con la lista de hoteles; aparece, por ejemplo, Casa Bocagrande (CTG, 3 estrellas).

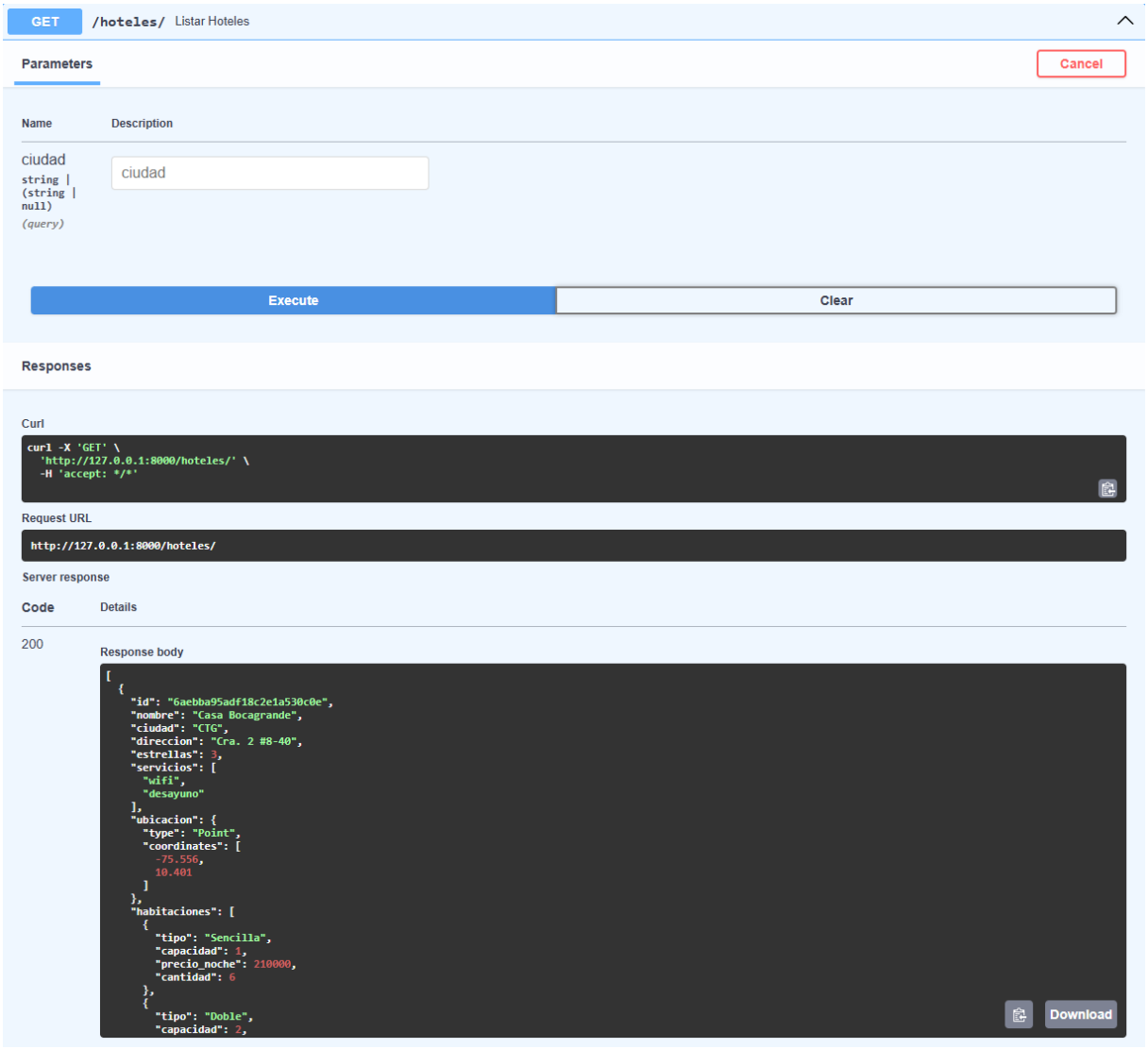


Figura 27: Prueba backend-05 en Swagger: listar hoteles.

5.3.2. backend-06. Registrar hotel

Método	POST
Ruta	`/hoteles/`
Colecciones	`hoteles`
Reglas en la base	Validador `$jsonSchema`

Descripción. Registra un hotel con nombre, ciudad IATA, dirección, estrellas (1 a 5) y de 1 a 4 habitaciones sin tipo repetido, cada una con capacidad y precio por noche.

Resultado de la prueba. 201 Created; se crea el hotel PRUEBA-20261104-Hotel (CTG, 3 estrellas) con habitación Doble de $200.000 la noche.

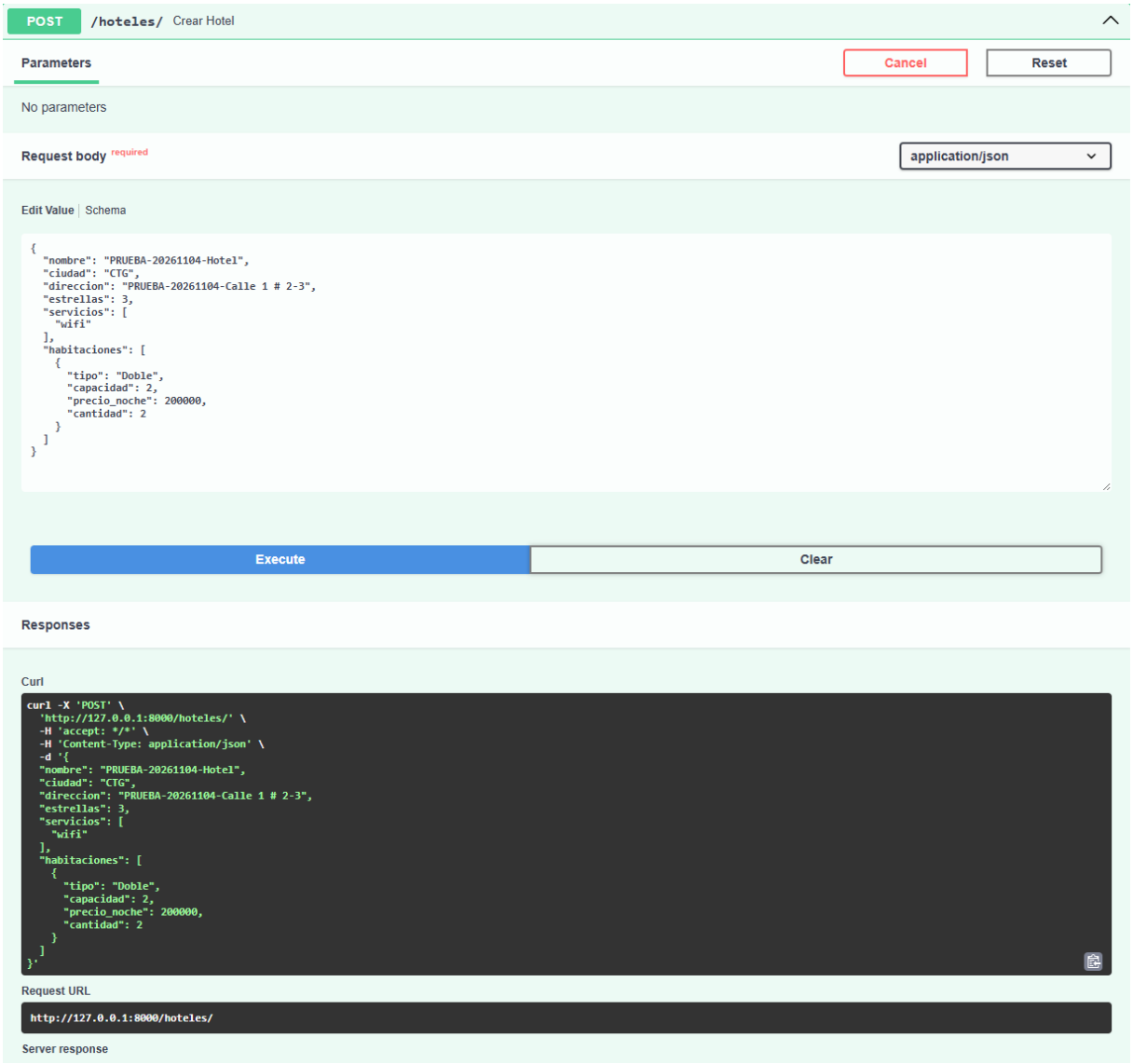


Figura 28: Prueba backend-06 en Swagger: registrar hotel.

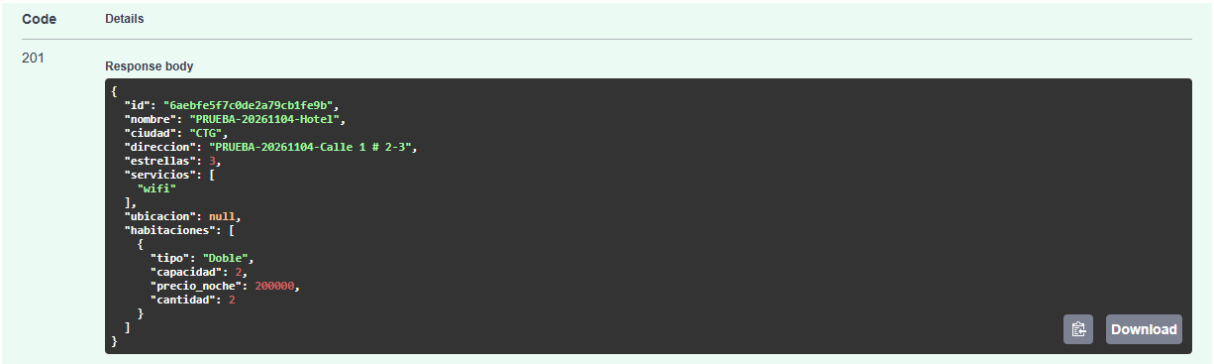


Figura 29: Prueba backend-06: respuesta 201 con el hotel creado.

5.4. Tours

Un tour tiene nombre, ciudad, descripción, duración en horas, qué incluye, punto de encuentro y varias salidas programadas, cada una con fecha, precio por persona y cupos.

5.4.1. backend-07. Listar tours

Método	GET
Ruta	`/tours/`
Colecciones	`tours`, `salidas`
Reglas en la base	Ninguna

Descripción. Devuelve el catálogo de tours. Admite el parámetro opcional `ciudad` (IATA).

Resultado de la prueba. 200 OK con la lista de tours; aparecen, por ejemplo, Ciudad Amurallada a pie (CTG, 3 horas) y Comuna 13 Graffitour (MDE, 4 horas).

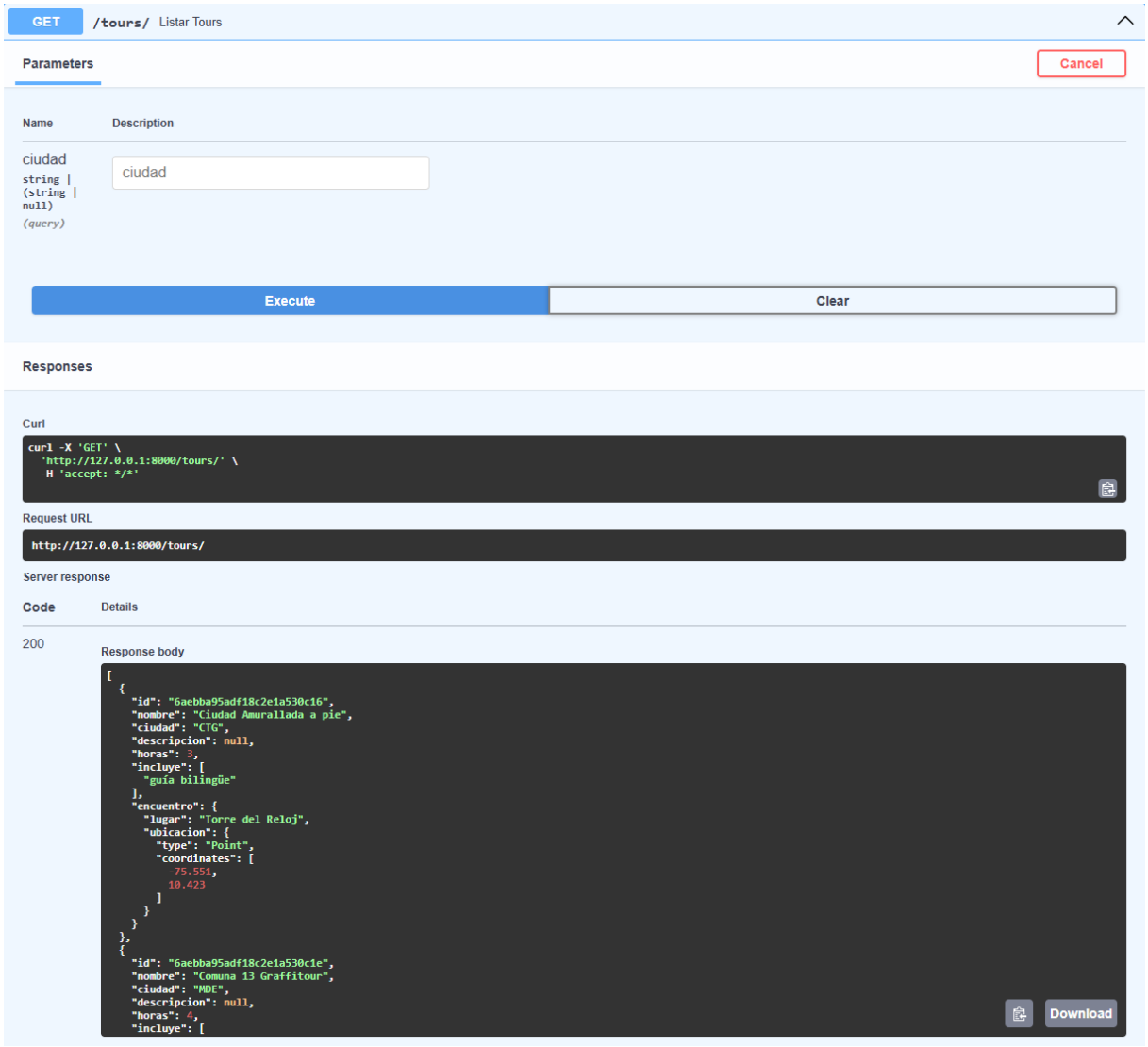


Figura 30: Prueba backend-07 en Swagger: listar tours.

5.4.2. backend-08. Registrar tour

Método	POST
Ruta	`/tours/`
Colecciones	`tours`
Reglas en la base	Ninguna

Descripción. Registra un tour con nombre y ciudad IATA; la descripción, las horas, lo que incluye y el punto de encuentro son opcionales. Las salidas con cupos se crean aparte.

Resultado de la prueba. 201 Created; se crea el tour PRUEBA-20261104-Tour (CTG, 2 horas).

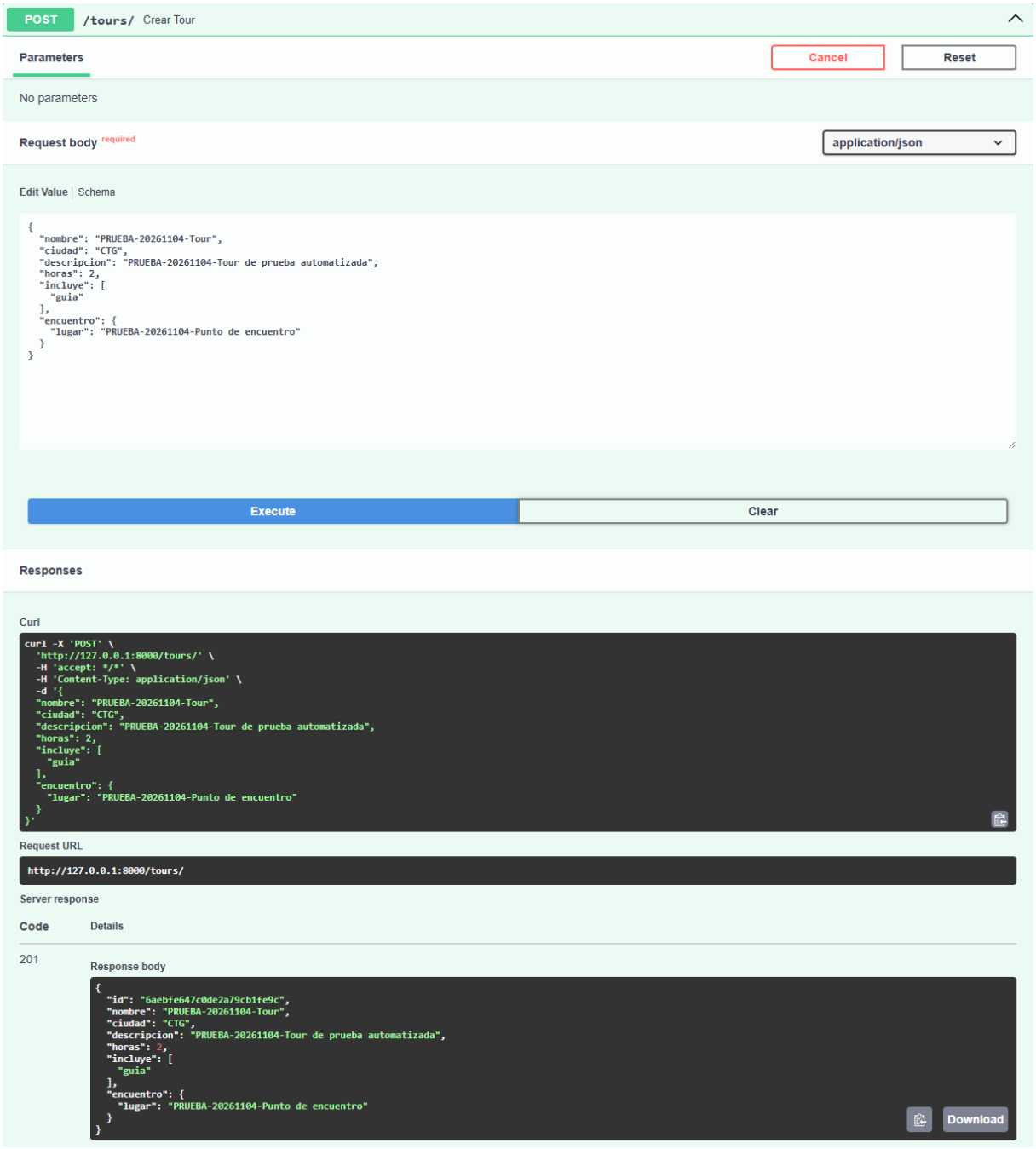


Figura 31: Prueba backend-08 en Swagger: registrar tour.

5.5. Paquetes

Un paquete combina productos (vuelos, noches de hotel, tours) con un porcentaje de descuento sobre la suma de los componentes y una ventana de vigencia. Guarda la referencia de cada componente y una copia de su nombre y precio: si el catálogo cambia después, el paquete no cambia.

5.5.1. backend-09. Listar paquetes

Método	GET
Ruta	`/paquetes/`
Colecciones	`paquetes`
Reglas en la base	Ninguna

Descripción. Devuelve el catálogo de paquetes con sus componentes y precio con descuento. Admite los parámetros opcionales `destino` (IATA) y `solo_vigentes` (por defecto true: solo paquetes dentro de su ventana de venta).

Resultado de la prueba. 200 OK; el paquete Cartagena 4 noches (descuento del 10 %) muestra sus componentes con la referencia y la copia del nombre y el precio de cada uno.

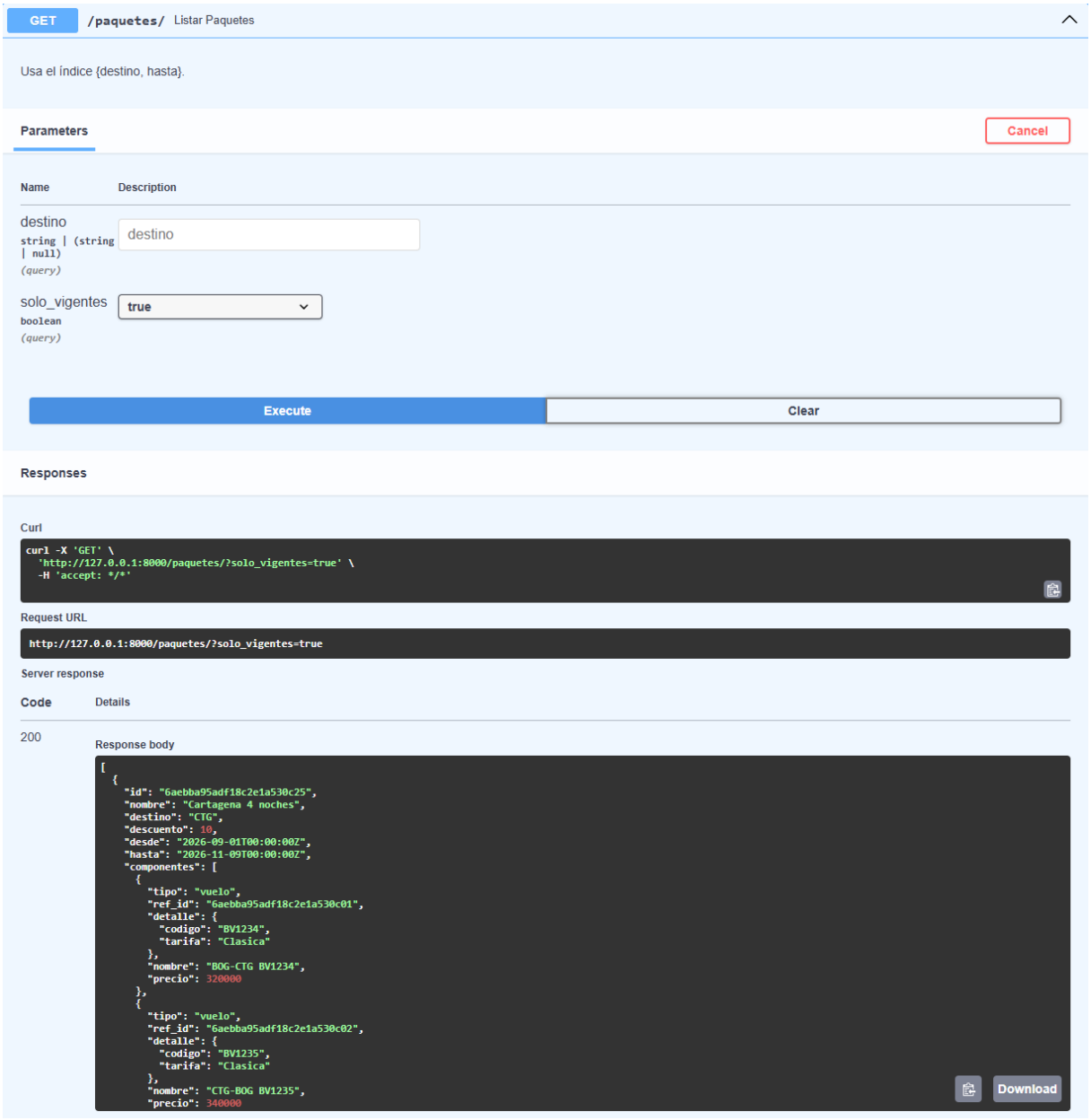


Figura 32: Prueba backend-09 en Swagger: listar paquetes vigentes.

5.5.2. backend-10. Registrar paquete

Método	POST
Ruta	`/paquetes/`
Colecciones	`paquetes`, `vuelos`, `hoteles`, `tours`
Reglas en la base	Ninguna

Descripción. Registra un paquete con nombre, destino IATA, descuento (0 a 100), ventana de venta (`desde`/`hasta`) y uno o más componentes. De cada componente se guarda la referencia y una copia de su nombre y precio vigente.

Resultado de la prueba. 201 Created; se crea el paquete PRUEBA-20261104-Paquete (CTG, descuento del 10 %, vigente del 2026-11-04 al 2027-01-19) con un componente vuelo en tarifa Basica, cuyo precio queda copiado en $180.000.

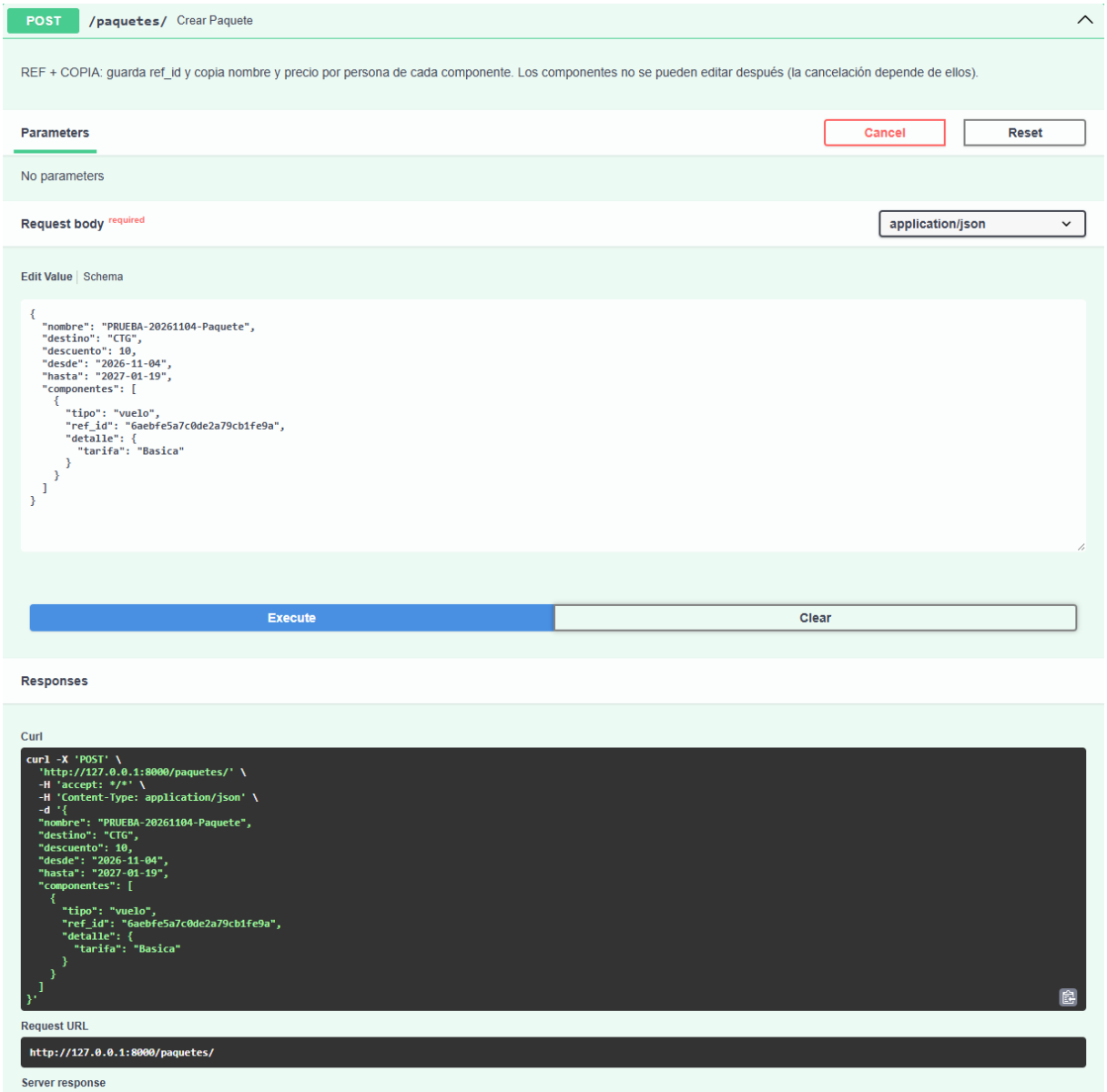


Figura 33: Prueba backend-10 en Swagger: registrar paquete.

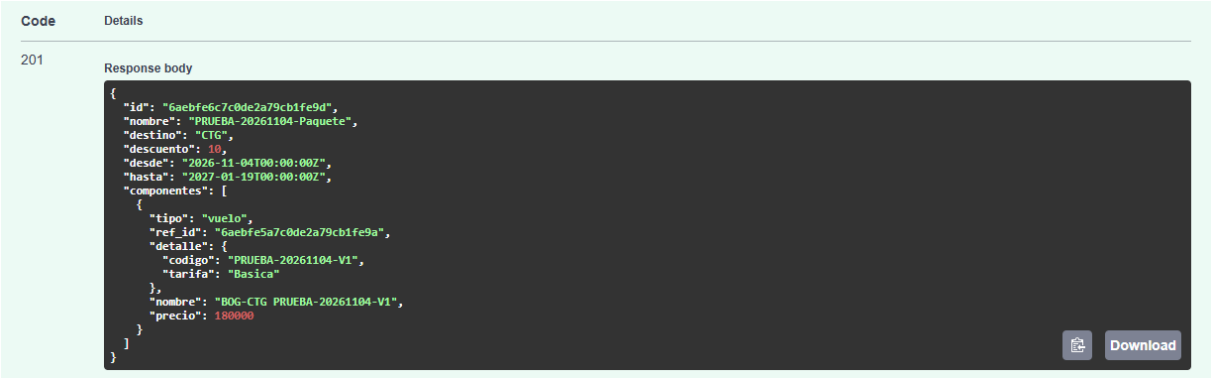


Figura 34: Prueba backend-10: respuesta 201 con el paquete creado y la copia del precio del componente.

5.6. Reservas

Una reserva pertenece a un cliente, la registra un asesor y contiene uno o varios ítems con su cantidad y el precio vigente al reservar. Guarda los pasajeros, admite pagos parciales y lleva el historial de estados con fecha y usuario. Al crearse descuenta el inventario y al cancelarse lo devuelve.

5.6.1. backend-11. Listar reservas

Método	GET
Ruta	`/reservas/`
Colecciones	`reservas`
Reglas en la base	Ninguna

Descripción. Devuelve la lista de reservas. Admite los parámetros opcionales `estado` (Pendiente, Pagada o Cancelada), `ciudad` (IATA, busca en los ítems), `cliente` (documento) y `limite`.

Resultado de la prueba. 200 OK con la lista de reservas; aparece, por ejemplo, la RV-261104-64F9 de Gloria Ospina en estado Pendiente.

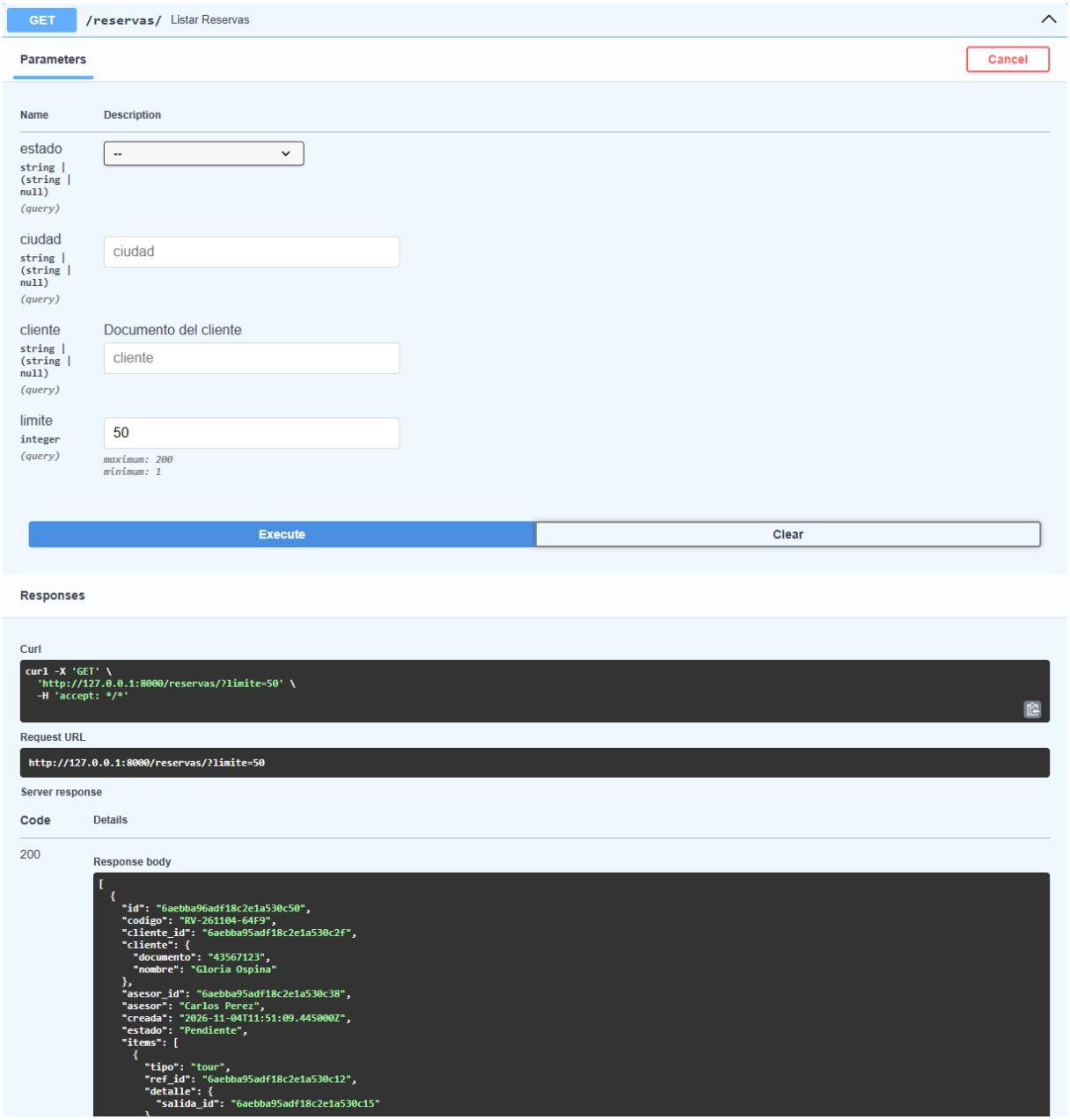


Figura 35: Prueba backend-11 en Swagger: listar reservas.

5.6.2. backend-12. Crear reserva

Método	POST
Ruta	`/reservas/`
Colecciones	`reservas`, `clientes`, `asesores`, `vuelos`
Reglas en la base	Escritura condicional atómica que descuenta el inventario solo si hay cupo (`$elemMatch` + `$inc`); índice único en `codigo`

Descripción. Crea una reserva con cliente, asesor, ítems y pasajeros. Antes de crearla se verifica la disponibilidad y el inventario se descuenta de inmediato; la reserva queda

Pendiente y cada ítem guarda el precio vigente en el momento de reservar. Si no hay cupo, la API responde 409; la evidencia 2 muestra el rechazo y el inventario sin cambios.

Resultado de la prueba. 201 Created; se crea la reserva RV-261105-4A4B del cliente de prueba con 1 silla en tarifa Basica del vuelo PRUEBA-20261104-V1, con el precio copiado en $180.000, total $180.000 e historial inicial en Pendiente registrado por el asesor AS-01. Las sillas libres de la tarifa bajan de 10 a 9.

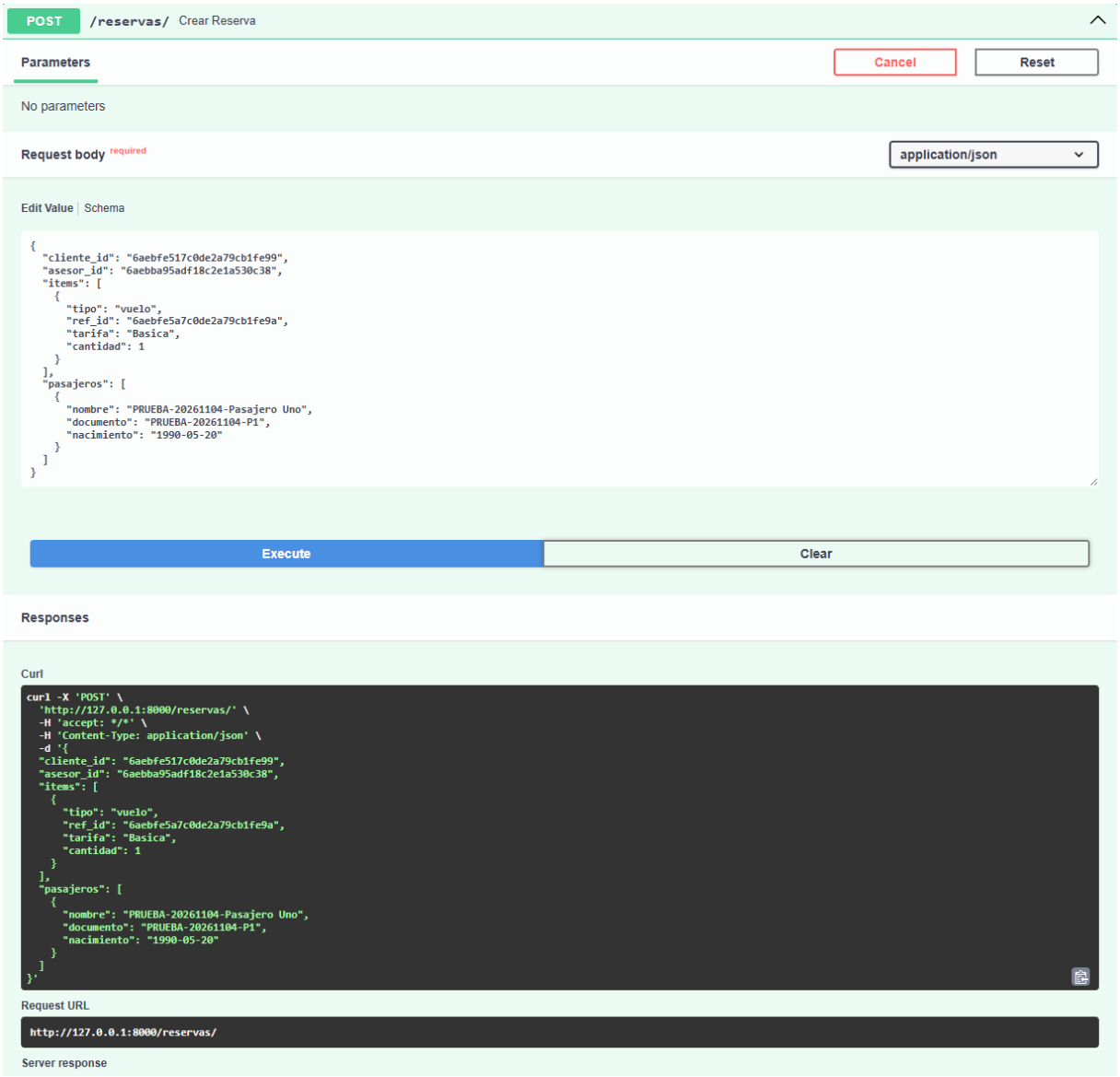


Figura 36: Prueba backend-12 en Swagger: crear reserva.

Code Details

201

Response body


```
{
  "id": "6aebfe747c0de2a79cb1fe9e",
  "codigo": "RV-261105-4A4B",
  "cliente_id": "6aebfe517c0de2a79cb1fe99",
  "cliente": {
    "documento": "PRUEBA-20261104-C1",
    "nombre": "PRUEBA-20261104-Cliente Uno"
  },
  "asesor_id": "6aebba95adf18c2e1a530c38",
  "asesor": "Carlos Perez",
  "creada": "2026-11-05T03:56:36.421000Z",
  "estado": "Pendiente",
  "items": [
    {
      "tipo": "vuelo",
      "ref_id": "6aebfe5a7c0de2a79cb1fe9a",
      "detalle": {
        "codigo": "PRUEBA-20261104-V1",
        "tarifa": "Basica"
      },
      "nombre": "BOG-CTG PRUEBA-20261104-V1",
      "ciudad": "CTG",
      "inicio": "2027-01-20T12:00:00Z",
      "fin": "2027-01-20T13:30:00Z",
      "cantidad": 1,
      "precio": 180000,
      "subtotal": 180000
    }
```


Download

Figura 37: Prueba backend-12: respuesta 201 con la reserva creada y el precio copiado en el ítem.

Code Details

201

Response body


```
      "ciudad": "CTG",
      "inicio": "2027-01-20T12:00:00Z",
      "fin": "2027-01-20T13:30:00Z",
      "cantidad": 1,
      "precio": 180000,
      "subtotal": 180000
    }
  ],
  "pasajeros": [
    {
      "nombre": "PRUEBA-20261104-Pasajero Uno",
      "documento": "PRUEBA-20261104-P1",
      "nacimiento": "1990-05-20T00:00:00Z"
    }
  ],
  "total": 180000,
  "pagado": 0,
  "saldo": 180000,
  "pagos": [],
  "historial": [
    {
      "antes": null,
      "despues": "Pendiente",
      "fecha": "2026-11-05T03:56:36.421000Z",
      "usuario": "AS-01"
    }
  ]
}
```


Download

Figura 38: Prueba backend-12: la reserva queda en estado Pendiente con su historial inicial.

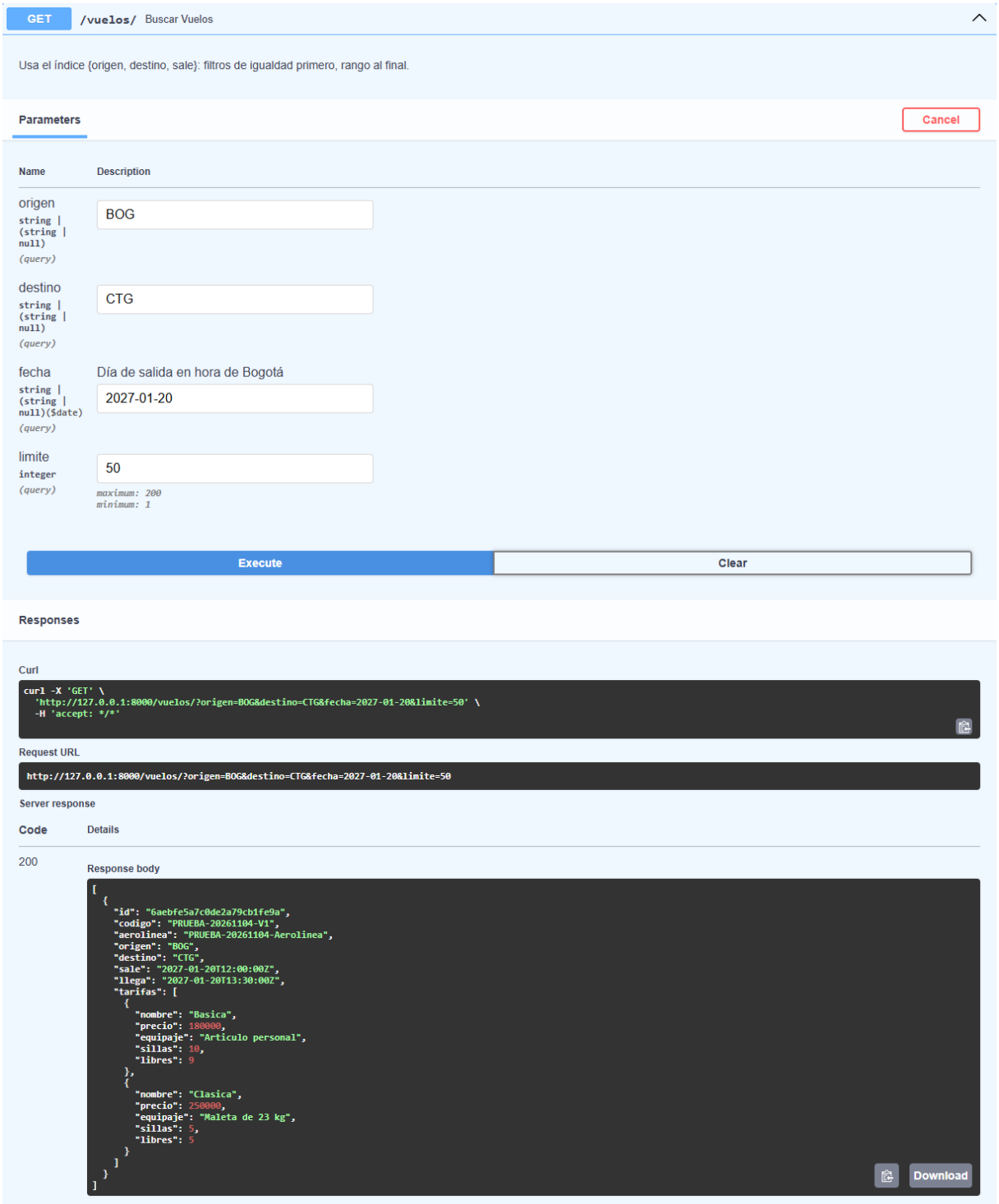


Figura 39: Prueba backend-12: el inventario del vuelo baja de 10 a 9 sillas libres en tarifa Basica.

5.6.3. backend-13. Obtener detalle de una reserva

Método	GET
Ruta	`/reservas/{codigo}`
Colecciones	`reservas`
Reglas en la base	Ninguna

Descripción. Devuelve el detalle completo de una reserva: ítems, pasajeros, pagos e historial de estados. Responde 404 si la reserva no existe.

Resultado de la prueba. 200 OK con el detalle de la reserva RV-261105-4A4B. Con el código inexistente RV-000000-0000 responde 404 ‘‘`Reserva no encontrado`’’.

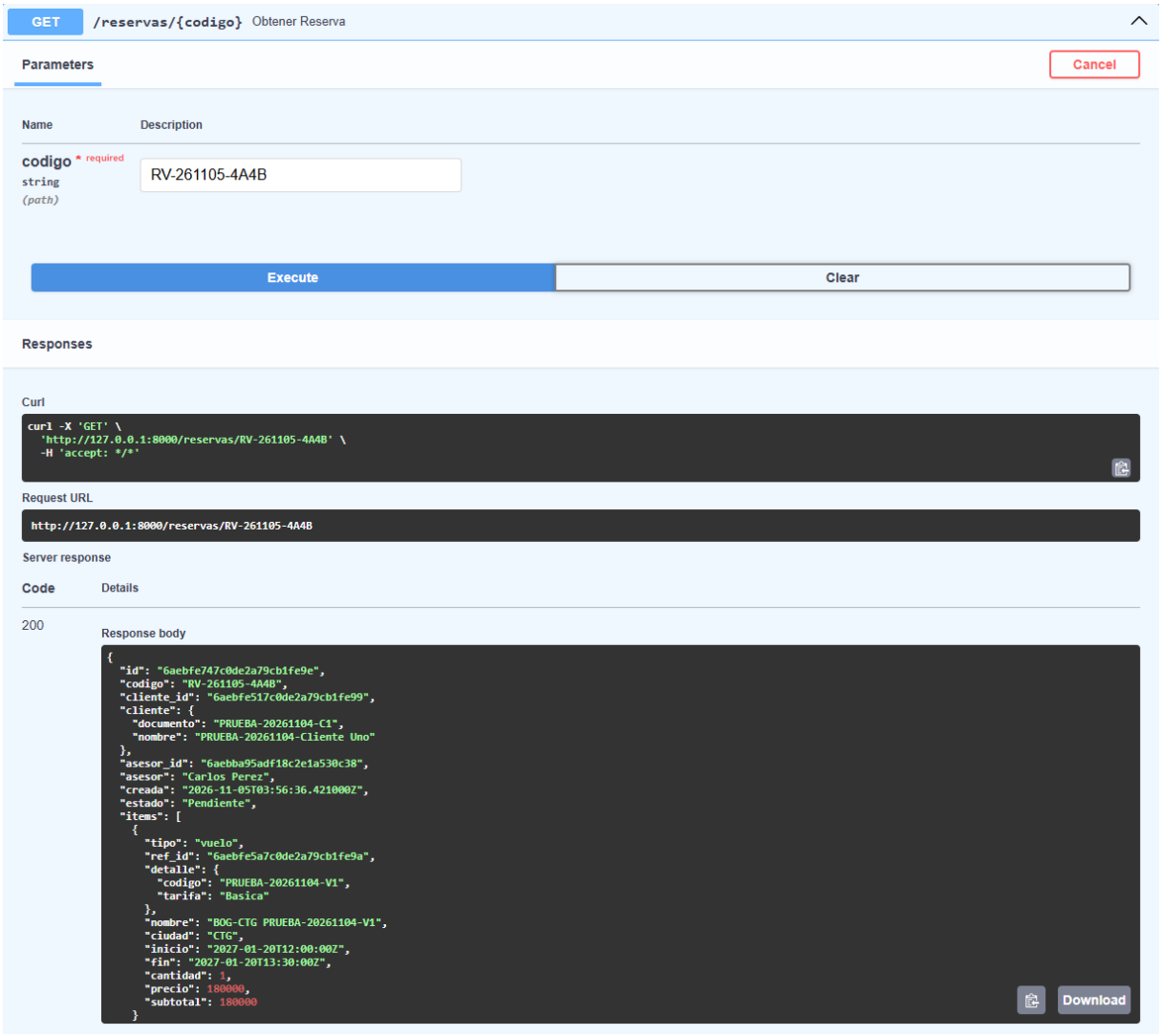


Figura 40: Prueba backend-13 en Swagger: detalle de la reserva.

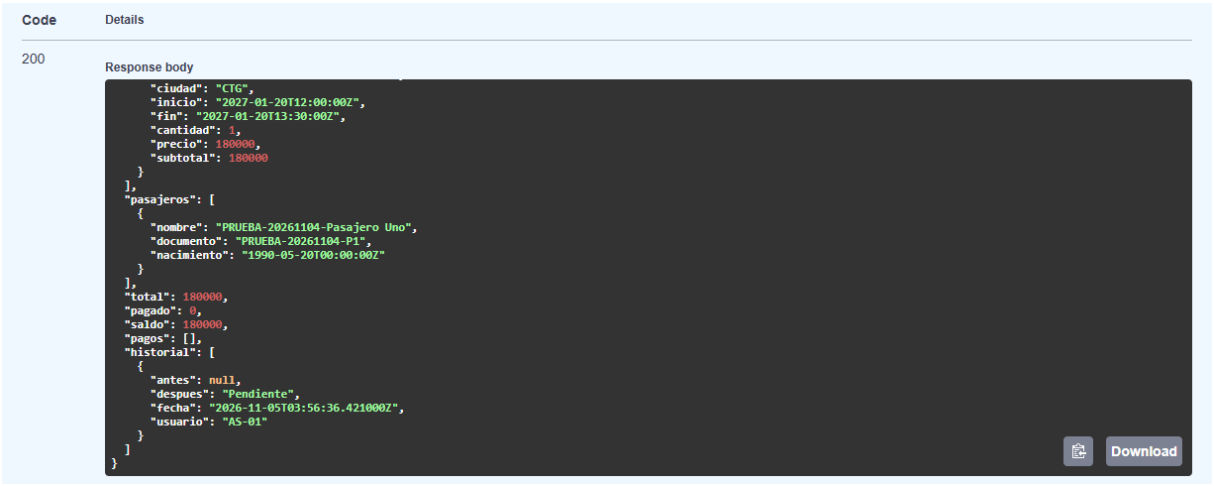


Figura 41: Prueba backend-13: el detalle incluye ítems, pasajeros, pagos e historial.

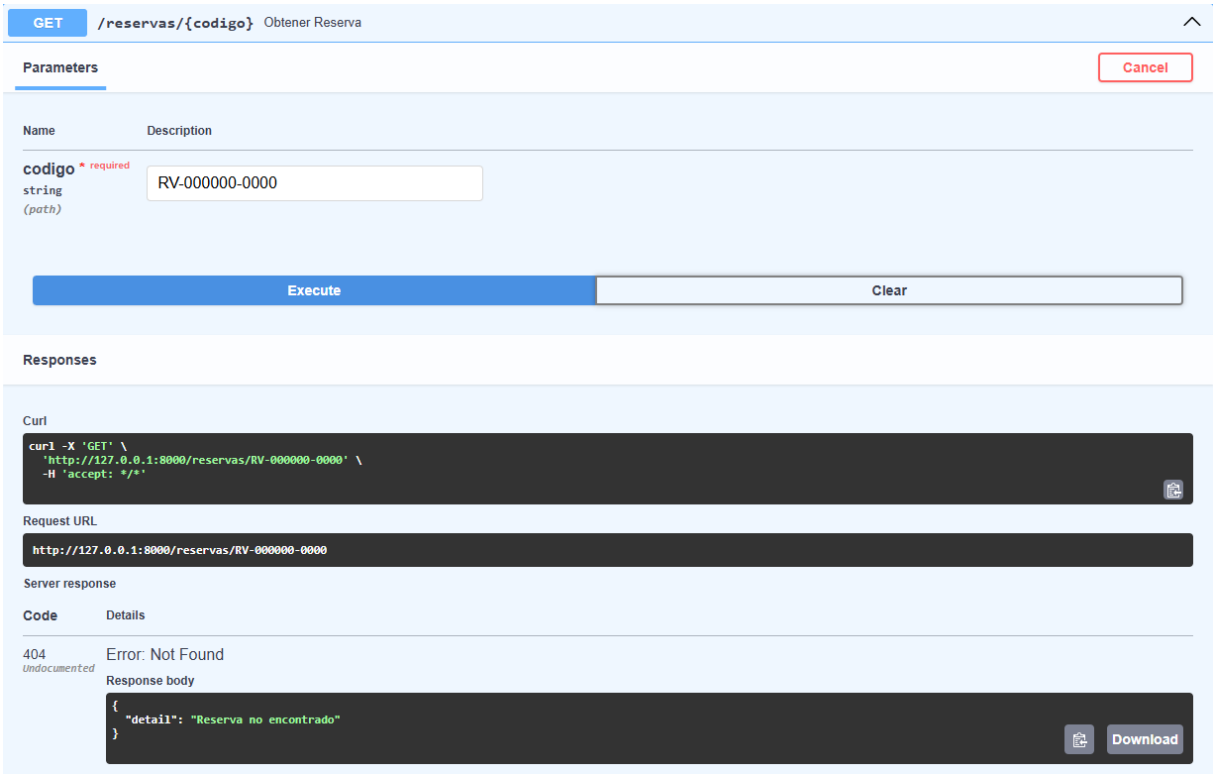


Figura 42: Prueba backend-13: un código inexistente responde 404.

5.6.4. backend-14. Registrar pago en una reserva

Método	POST
Ruta	`/reservas/{codigo}/pagos`
Colecciones	`reservas`, `asesores`
Reglas en la base	Escritura atómica que agrega el pago y actualiza lo pagado (`$push` + `$inc`)

Descripción. Registra un pago con `valor` (entero en pesos, mayor que 0 y sin superar el saldo pendiente), `medio` (Efectivo, Tarjeta, Transferencia o PSE) y `usuario` (el código de un asesor activo). Cuando los pagos cubren el total, la reserva pasa a Pagada y el cambio queda en el historial. No se aceptan pagos en reservas canceladas.

Resultado de la prueba. 200 OK; un pago de $180.000 en Efectivo registrado por AS-01 cubre el total de la reserva RV-261105-4A4B: el estado pasa a Pagada, lo pagado queda en $180.000 con saldo 0 y el historial agrega el cambio Pendiente → Pagada con fecha y usuario.

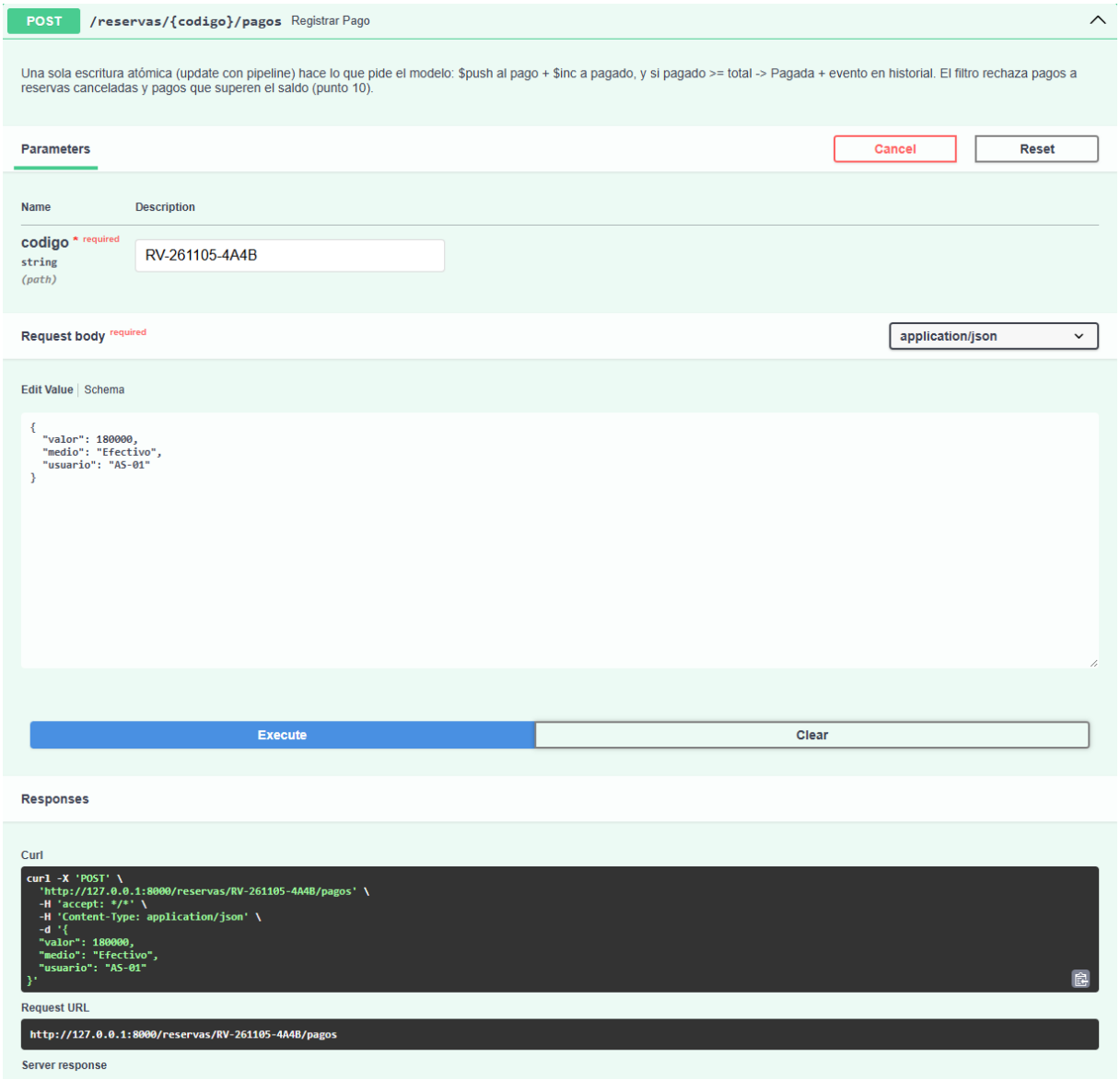


Figura 43: Prueba backend-14 en Swagger: registrar pago.

Code Details

200

Response body


```
{
  "id": "6aebfe747c0de2a79cb1fe9e",
  "codigo": "RV-261105-4A4B",
  "cliente_id": "6aebfe517c0de2a79cb1fe99",
  "cliente": {
    "documento": "PRUEBA-20261104-C1",
    "nombre": "PRUEBA-20261104-Cliente Uno"
  },
  "asesor_id": "6aebba95adf18c2e1a530c38",
  "asesor": "Carlos Perez",
  "creada": "2026-11-05T03:56:36.421000Z",
  "estado": "Pagada",
  "items": [
    {
      "tipo": "vuelo",
      "ref_id": "6aebfe5a7c0de2a79cb1fe9a",
      "detalle": {
        "codigo": "PRUEBA-20261104-V1",
        "tarifa": "Basica"
      },
      "nombre": "BOG-CTG PRUEBA-20261104-V1",
      "ciudad": "CTG",
      "inicio": "2027-01-20T12:00:00Z",
      "fin": "2027-01-20T13:30:00Z",
      "cantidad": 1,
      "precio": 180000,
      "subtotal": 180000
    }
```


Download

Figura 44: Prueba backend-14: respuesta 200 con la reserva en estado Pagada.

Code Details

200

Response body


```
      "precio": 180000,
      "subtotal": 180000
    }
  ],
  "pasajeros": [
    {
      "nombre": "PRUEBA-20261104-Pasajero Uno",
      "documento": "PRUEBA-20261104-P1",
      "nacimiento": "1990-05-20T00:00:00Z"
    }
  ],
  "total": 180000,
  "pagado": 180000,
  "saldo": 0,
  "pagos": [
    {
      "fecha": "2026-11-05T03:56:47.152000Z",
      "valor": 180000,
      "medio": "Efectivo"
    }
  ],
  "historial": [
    {
      "antes": null,
      "despues": "Pendiente",
      "fecha": "2026-11-05T03:56:36.421000Z",
      "usuario": "AS-01"
    },
    {
```


Download

Figura 45: Prueba backend-14: el pago queda registrado con fecha, valor y medio.

Code Details

200

Response body


```
      "nacimiento": "1990-05-20T00:00:00Z"
    }
  ],
  "total": 180000,
  "pagado": 180000,
  "saldo": 0,
  "pagos": [
    {
      "fecha": "2026-11-05T03:56:47.152000Z",
      "valor": 180000,
      "medio": "Efectivo"
    }
  ],
  "historial": [
    {
      "antes": null,
      "despues": "Pendiente",
      "fecha": "2026-11-05T03:56:36.421000Z",
      "usuario": "AS-01"
    },
    {
      "antes": "Pendiente",
      "despues": "Pagada",
      "fecha": "2026-11-05T03:56:47.152000Z",
      "usuario": "AS-01"
    }
  ]
}
```


Download

Figura 46: Prueba backend-14: el historial agrega el cambio de Pendiente a Pagada con el usuario AS-01.